School of Engineering and Technology

Private University Estd. in Karnataka State by Act No. 45 of 2013

4CSPL1012

PROBLEM SOLVING USING PYTHON

(Intensive Coding Class)

Laboratory Manual

Created by:

Dr. Pandava Sudharshan Babu

and

Python Faculty Team

CMR University, Bengaluru

Academic Year 2025–2026

Contents

1 Lab 1: Basics – print(), input(), Expressions **4**
1.1 Objective . . . 4
1.2 Theory . . . 4
1.3 Programs . . . 4
1.3.1 Basic Programs . . . 5
1.3.2 Medium Programs . . . 9
1.3.3 Advanced Programs . . . 13
1.4 Learning Outcomes . . . 16

2 Lab 2: Decision Constructs (if, elif, nested if) **17**
2.1 Objective . . . 17
2.2 Theory . . . 17
2.3 Programs . . . 17
2.3.1 Basic Programs . . . 17
2.3.2 Medium Programs . . . 21
2.3.3 Advanced Programs . . . 27
2.4 Learning Outcomes . . . 30

3 Lab 3 & 4: Loops (for, while, break, continue, pass) **32**
3.1 Objective . . . 32
3.2 Theory . . . 32
3.3 Programs . . . 32
3.3.1 Basic Programs . . . 32
3.3.2 Medium Programs . . . 37
3.3.3 Advanced Programs . . . 41
3.4 Learning Outcomes . . . 45

4 Lab 5: Strings, Functions, and Recursion **46**
4.1 Objective . . . 46
4.2 Theory . . . 46
4.3 Programs . . . 46
4.3.1 Basic Programs . . . 46
4.3.2 Medium Programs . . . 50
4.3.3 Advanced Programs . . . 53
4.4 Learning Outcomes . . . 55

5 Lab 6: Lists and Tuples (Mutable vs Immutable) **56**
5.1 Objective . . . 56
5.2 Theory . . . 56
5.3 Programs . . . 56
5.3.1 Basic Programs . . . 56
5.3.2 Medium Programs . . . 59
5.3.3 Advanced Programs . . . 62
5.4 Learning Outcomes . . . 64

6 Lab 7: Sets and Dictionaries **65**
6.1 Objective . . . 65

6.2 Theory . . . 65
6.3 Programs . . . 65
6.3.1 Basic Programs . . . 65
6.3.2 Medium Programs . . . 68
6.3.3 Advanced Programs . . . 70
6.4 Learning Outcomes . . . 72

7 Lab 8 & 9: Exception Handling 73
7.1 Objective . . . 73
7.2 Theory . . . 73
7.3 Programs . . . 73
7.3.1 Basic Programs . . . 73
7.3.2 Medium Programs . . . 75
7.3.3 Advanced Programs . . . 78
7.4 Learning Outcomes . . . 81

8 Lab 10: Classes and Objects 82
8.1 Objective . . . 82
8.2 Theory . . . 82
8.3 Programs . . . 82
8.3.1 Basic Programs . . . 82
8.3.2 Medium Programs . . . 86
8.3.3 Advanced Programs . . . 90
8.4 Learning Outcomes . . . 93

9 Lab 11: Static Polymorphism 95
9.1 Objective . . . 95
9.2 Theory . . . 95
9.3 Programs . . . 95
9.3.1 Basic Programs . . . 95
9.3.2 Medium Programs . . . 97
9.3.3 Advanced Programs . . . 101
9.4 Learning Outcomes . . . 103

10 Lab 12: Inheritance 104
10.1 Objective . . . 104
10.2 Theory . . . 104
10.3 Programs . . . 104
10.3.1 Basic Programs . . . 104
10.3.2 Medium Programs . . . 107
10.3.3 Advanced Programs . . . 112
10.4 Learning Outcomes . . . 114

11 Lab 13 & 14: Abstraction and Encapsulation 115
11.1 Objective . . . 115
11.2 Theory . . . 115
11.3 Programs . . . 116
11.3.1 Basic Programs . . . 116
11.3.2 Medium Programs . . . 121

11.3.3 Advanced Programs 127
11.4 Learning Outcomes 131

1 Lab 1: Basics – print(), input(), Expressions

1.1 Objective

To understand the fundamental building blocks of Python programming including the `print()` function for output, the `input()` function for user interaction, and basic arithmetic expressions for computation.

1.2 Theory

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. It emphasizes code readability with its notable use of significant indentation. Python is widely used for web development, data analysis, artificial intelligence, scientific computing, and automation.

The print() Function: The `print()` function is one of the most fundamental built-in functions in Python. It is used to display output on the console (standard output). The function can accept multiple arguments separated by commas. Two important parameters of `print()` are:

- `sep` – Specifies the separator between multiple arguments. The default separator is a single space character.
- `end` – Specifies what to print at the end of the output. The default is a newline character (`\n`).

The input() Function: The `input()` function allows a Python program to accept data from the user during runtime. It always returns the input as a string, so type conversion (such as `int()` or `float()`) is often needed when numerical data is expected. The function can take an optional string argument that serves as a prompt message displayed to the user.

Expressions: An expression in Python is a combination of values, variables, operators, and function calls that the interpreter evaluates to produce a result. Python supports several types of operators:

- **Arithmetic Operators:** `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division), `//` (floor division), `%` (modulus), `**` (exponentiation).
- **Assignment Operators:** `=`, `+=`, `-=`, `*=`, `/=`, etc.
- **Comparison Operators:** `==`, `!=`, `>`, `<`, `>=`, `<=`.

Python follows the standard mathematical order of operations (PEMDAS/BODMAS) when evaluating expressions. Variables in Python are dynamically typed, meaning a variable's type is determined at runtime based on the value assigned to it. The `eval()` function can be used to evaluate a string as a Python expression, which is useful for creating dynamic calculators but should be used cautiously in production code due to security concerns.

1.3 Programs

1.3.1 Basic Programs

Program 1: Print Name, Age, and City

Problem Statement: Write a Python program to print the user's name, age, and city on the console.

Algorithm:

Step 1: Start the program.
Step 2: Assign the value "Alice" to a variable named `name`.
Step 3: Assign the value 20 to a variable named `age`.
Step 4: Assign the value "Bengaluru" to a variable named `city`.
Step 5: Use the `print()` function to display each variable.
Step 6: End the program.

```python
# Program 1: Print Name, Age, and City
name = "Alice"
age = 20
city = "Bengaluru"

print("Name:", name)
print("Age:", age)
print("City:", city)
```

Sample Input: None (values are hardcoded)

Sample Output

```
Name: Alice
Age: 20
City: Bengaluru
```

Explanation of Output:

1. The program creates three variables: `name` (string), `age` (integer), and `city` (string).
2. Each `print()` statement displays a label followed by the variable value.
3. The comma in `print()` automatically adds a space between the label and the value.
4. Each `print()` call outputs on a new line by default.

Note:

In Python, variables do not need explicit type declarations. The type is inferred from the assigned value. This is called **dynamic typing**.

Program 2: Print Multiple Values Using sep Parameter

Problem Statement: Write a Python program to print multiple values separated by a custom separator using the `sep` parameter.

Algorithm:

Step 1: Start the program.
Step 2: Use `print()` with multiple string arguments.
Step 3: Set the `sep` parameter to different separators such as "-", ",", and " | ".
Step 4: Observe the output with different separators.
Step 5: End the program.

```python
# Program 2: Using sep parameter in print()
print("Python", "Java", "C++", sep=", ")
print("01", "01", "2026", sep="-")
print("Alice", "Bob", "Charlie", sep=" | ")
print("A", "B", "C", "D", sep="->")
```

Sample Input: None (values are hardcoded)

Sample Output

```
Python, Java, C++
01-01-2026
Alice | Bob | Charlie
A->B->C->D
```

Explanation of Output:

1. The first `print()` uses "`, `" as separator, producing a comma-separated list.
2. The second `print()` uses "`-`" as separator, formatting the output like a date.
3. The third `print()` uses "` | `" as separator, creating a pipe-delimited output.
4. The fourth `print()` uses "`->`" as separator, showing a chain-like format.

Program 3: Demonstrate end Parameter

Problem Statement: Write a Python program to demonstrate the use of the `end` parameter in the `print()` function.

Algorithm:

Step 1: Start the program.
Step 2: Use `print()` with the `end` parameter set to a space.
Step 3: Use `print()` with the `end` parameter set to custom strings.
Step 4: Observe that outputs continue on the same line.
Step 5: End the program.

```python
# Program 3: Demonstrate end parameter
print("Hello", end=" ")
print("World!")

print("Loading", end="...")
```

```python
print("Done!")

print("A", end=" -> ")
print("B", end=" -> ")
print("C")
```

Sample Input: None

Sample Output

```
Hello World!
Loading...Done!
A -> B -> C
```

Explanation of Output:

1. By default, `print()` ends with a newline. Setting `end=" "` replaces the newline with a space, so the next `print()` continues on the same line.
2. "Loading" is followed by "..." (no newline), and "Done!" follows immediately.
3. The chain "A -> B -> C" is created by using `end=" -> "` for the first two prints.

Program 4: Take Input and Print It

Problem Statement: Write a Python program to accept the user's name as input and display a greeting message.

Algorithm:

Step 1: Start the program.
Step 2: Prompt the user to enter their name using `input()`.
Step 3: Store the input in a variable `name`.
Step 4: Display a greeting message using the name.
Step 5: End the program.

```python
# Program 4: Take input and print it
name = input("Enter your name: ")
print("Hello,", name + "!")
print("Welcome to Python Programming.")
```

Sample Input:

`Enter your name: Sudharshan`

Sample Output

```
Enter your name: Sudharshan
Hello, Sudharshan!
Welcome to Python Programming.
```

Explanation of Output:

1. The `input()` function displays the prompt "Enter your name: " and waits for user input.
2. The user types "Sudharshan" and presses Enter.
3. The input is stored as a string in the variable `name`.
4. The `print()` function concatenates the name with "!" using the `+` operator and displays the greeting.

Program 5: Sum of Two Numbers

Problem Statement: Write a Python program to accept two numbers from the user and display their sum.

Algorithm:

Step 1: Start the program.
Step 2: Accept the first number from the user and convert it to integer.
Step 3: Accept the second number from the user and convert it to integer.
Step 4: Calculate the sum of the two numbers.
Step 5: Display the result.
Step 6: End the program.

```python
# Program 5: Sum of two numbers
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))
total = num1 + num2
print("Sum of", num1, "and", num2, "is:", total)
```

Sample Input:

```
Enter first number: 15
Enter second number: 25
```

Sample Output

```
Enter first number: 15
Enter second number: 25
Sum of 15 and 25 is: 40
```

Explanation of Output:

1. `input()` returns a string, so `int()` converts it to an integer.
2. The user enters 15 and 25.
3. The sum is calculated as $15 + 25 = 40$.
4. The result is displayed using `print()` with multiple arguments.

Note:

If the user enters a non-numeric value (e.g., "abc"), the program will raise a `ValueError`. Exception handling (covered in Lab 8) can be used to handle such cases.

1.3.2 Medium Programs

Program 6: Swap Two Numbers

Problem Statement: Write a Python program to swap the values of two variables and display the result before and after swapping.

Algorithm:

Step 1: Start the program.
Step 2: Accept two numbers from the user.
Step 3: Display the values before swapping.
Step 4: Swap the values using Python's tuple unpacking (simultaneous assignment).
Step 5: Display the values after swapping.
Step 6: End the program.

```python
# Program 6: Swap two numbers
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

print("Before swapping: a =", a, ", b =", b)

# Swap using tuple unpacking
a, b = b, a

print("After swapping: a =", a, ", b =", b)
```

Sample Input:

```
Enter first number: 10
Enter second number: 20
```

Sample Output

```
Enter first number: 10
Enter second number: 20
Before swapping: a = 10 , b = 20
After swapping: a = 20 , b = 10
```

Explanation of Output:

1. The user enters $a = 10$ and $b = 20$.
2. Before swapping, a is 10 and b is 20.
3. Python's simultaneous assignment `a, b = b, a` swaps values without a temporary variable.
4. After swapping, a becomes 20 and b becomes 10.

Note:

In many other languages (like C or Java), swapping requires a temporary variable. Python's tuple unpacking makes this elegant and concise.

Program 7: Area of Circle

Problem Statement: Write a Python program to calculate the area of a circle given its radius.

Algorithm:

Step 1: Start the program.
Step 2: Accept the radius from the user as a floating-point number.
Step 3: Define the value of π as 3.14159.
Step 4: Calculate the area using the formula: $A = \pi r^2$.
Step 5: Display the result rounded to 2 decimal places.
Step 6: End the program.

```python
# Program 7: Area of circle
radius = float(input("Enter the radius of the circle: "))
pi = 3.14159
area = pi * radius ** 2
print("Area of the circle with radius", radius, "is:", round(area
    , 2))
```

Sample Input:

```
Enter the radius of the circle: 7
```

Sample Output

```
Enter the radius of the circle: 7
Area of the circle with radius 7.0 is: 153.94
```

Explanation of Output:

1. The radius is entered as 7 and converted to float (7.0).
2. The area is computed as $3.14159 \times 7^2 = 3.14159 \times 49 = 153.93791$.
3. The `round()` function rounds the result to 2 decimal places: 153.94.

Program 8: Temperature Conversion

Problem Statement: Write a Python program to convert temperature from Celsius to Fahrenheit and vice versa.

Algorithm:

Step 1: Start the program.
Step 2: Accept a temperature in Celsius from the user.
Step 3: Convert to Fahrenheit using: $F = \frac{9}{5} \times C + 32$.
Step 4: Accept a temperature in Fahrenheit from the user.
Step 5: Convert to Celsius using: $C = \frac{5}{9} \times (F - 32)$.
Step 6: Display both results.
Step 7: End the program.

```python
# Program 8: Temperature conversion
celsius = float(input("Enter temperature in Celsius: "))
fahrenheit = (9/5) * celsius + 32
print(celsius, "^^b0C =", round(fahrenheit, 2), "^^b0F")

fahrenheit2 = float(input("Enter temperature in Fahrenheit: "))
celsius2 = (5/9) * (fahrenheit2 - 32)
print(fahrenheit2, "^^b0F =", round(celsius2, 2), "^^b0C")
```

Sample Input:

```
Enter temperature in Celsius: 37
Enter temperature in Fahrenheit: 212
```

Sample Output

```
Enter temperature in Celsius: 37
37.0 ^^b0C = 98.6 ^^b0F
Enter temperature in Fahrenheit: 212
212.0 ^^b0F = 100.0 ^^b0C
```

Explanation of Output:

1. 37°C is normal human body temperature. Converting: $\frac{9}{5} \times 37 + 32 = 66.6 + 32 = 98.6$°F.
2. 212°F is the boiling point of water. Converting: $\frac{5}{9} \times (212 - 32) = \frac{5}{9} \times 180 = 100$°C.

Program 9: Discount Calculation

Problem Statement: Write a Python program to calculate the final price after applying a discount percentage to the original price.

Algorithm:

Step 1: Start the program.
Step 2: Accept the original price from the user.
Step 3: Accept the discount percentage from the user.
Step 4: Calculate discount amount: discount $= \frac{\text{price} \times \text{rate}}{100}$.
Step 5: Calculate final price: final = price − discount.
Step 6: Display the discount amount and final price.
Step 7: End the program.

```python
# Program 9: Discount calculation
price = float(input("Enter original price: "))
discount_rate = float(input("Enter discount percentage: "))

discount_amount = (price * discount_rate) / 100
final_price = price - discount_amount

print("Original Price: Rs.", price)
```

```
print("Discount (", discount_rate, "%): Rs.", round(
    discount_amount, 2))
print("Final Price: Rs.", round(final_price, 2))
```

Sample Input:

```
Enter original price: 1500
Enter discount percentage: 20
```

Sample Output

```
Enter original price: 1500
Enter discount percentage: 20
Original Price: Rs. 1500.0
Discount ( 20.0 %): Rs. 300.0
Final Price: Rs. 1200.0
```

Explanation of Output:

1. Original price is Rs. 1500 and the discount rate is 20%.
2. Discount amount: $\frac{1500 \times 20}{100} = 300$.
3. Final price: $1500 - 300 = 1200$.

Program 10: Average of 3 Numbers

Problem Statement: Write a Python program to calculate the average of three numbers entered by the user.

Algorithm:

Step 1: Start the program.
Step 2: Accept three numbers from the user.
Step 3: Calculate the sum of the three numbers.
Step 4: Divide the sum by 3 to get the average.
Step 5: Display the result.
Step 6: End the program.

```
# Program 10: Average of 3 numbers
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
num3 = float(input("Enter third number: "))

total = num1 + num2 + num3
average = total / 3

print("Sum:", total)
print("Average:", round(average, 2))
```

Sample Input:

```
Enter first number: 85
Enter second number: 90
Enter third number: 78
```

Sample Output

```
Enter first number: 85
Enter second number: 90
Enter third number: 78
Sum: 253.0
Average: 84.33
```

Explanation of Output:

1. Three numbers are entered: 85, 90, and 78.
2. Sum: $85 + 90 + 78 = 253$.
3. Average: $\frac{253}{3} = 84.3333...$, rounded to 84.33.

1.3.3 Advanced Programs

Program 11: Expression Evaluator Using eval()

Problem Statement: Write a Python program that accepts a mathematical expression as a string from the user and evaluates it using the `eval()` function.

Algorithm:

Step 1: Start the program.
Step 2: Prompt the user to enter a mathematical expression as a string.
Step 3: Use the `eval()` function to evaluate the expression.
Step 4: Display the expression and its result.
Step 5: Repeat for multiple expressions to demonstrate capability.
Step 6: End the program.

```python
# Program 11: Expression evaluator using eval()
print("=== Expression Evaluator ===")
print("Enter a mathematical expression to evaluate.")
print("Examples: 2+3*4, (10-3)**2, 100/3")

expression = input("\nEnter expression: ")
result = eval(expression)
print("Expression:", expression)
print("Result:", result)

# Additional examples
print("\n--- More Examples ---")
expr1 = "2 ** 10"
print(expr1, "=", eval(expr1))

expr2 = "(50 + 30) * 2 - 10"
print(expr2, "=", eval(expr2))
```

```python
expr3 = "100 // 3"
print(expr3, "=", eval(expr3))

expr4 = "17 % 5"
print(expr4, "=", eval(expr4))
```

Sample Input:

```
Enter expression: 5 + 3 * 2
```

Sample Output

```
=== Expression Evaluator ===
Enter a mathematical expression to evaluate.
Examples: 2+3*4, (10-3)**2, 100/3

Enter expression: 5 + 3 * 2
Expression: 5 + 3 * 2
Result: 11

--- More Examples ---
2 ** 10 = 1024
(50 + 30) * 2 - 10 = 150
100 // 3 = 33
17 % 5 = 2
```

Explanation of Output:

1. The user enters "5 + 3 * 2". Due to operator precedence, multiplication is done first: $3 \times 2 = 6$, then addition: $5 + 6 = 11$.
2. $2^{10} = 1024$ (exponentiation).
3. $(50 + 30) \times 2 - 10 = 80 \times 2 - 10 = 160 - 10 = 150$.
4. $100 \div 3 = 33$ (floor division discards the remainder).
5. $17 \mod 5 = 2$ (remainder when 17 is divided by 5).

Note:

Security Warning: The `eval()` function executes arbitrary Python code. Never use it with untrusted input in production applications, as it can execute malicious code. It is used here for educational purposes only.

Program 12: Mini Calculator Using Expressions

Problem Statement: Write a Python program that acts as a mini calculator, performing addition, subtraction, multiplication, division, floor division, modulus, and exponentiation on two user-provided numbers.

Algorithm:

Step 1: Start the program.
Step 2: Accept two numbers from the user.
Step 3: Perform all arithmetic operations on the two numbers.
Step 4: Display the results of each operation.
Step 5: Handle division by zero scenario.
Step 6: End the program.

```python
# Program 12: Mini calculator using expressions
print("===== Mini Calculator =====")
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print("\n--- Results ---")
print("Addition:       ", num1, "+", num2, "=", num1 + num2)
print("Subtraction:    ", num1, "-", num2, "=", num1 - num2)
print("Multiplication: ", num1, "*", num2, "=", num1 * num2)

if num2 != 0:
    print("Division:       ", num1, "/", num2, "=", round(num1 /
        num2, 4))
    print("Floor Division: ", num1, "//", num2, "=", num1 // num2
        )
    print("Modulus:        ", num1, "%", num2, "=", num1 % num2)
else:
    print("Division:        Cannot divide by zero!")
    print("Floor Division:  Cannot divide by zero!")
    print("Modulus:         Cannot divide by zero!")

print("Exponentiation: ", num1, "**", num2, "=", num1 ** num2)
```

Sample Input:

```
Enter first number: 17
Enter second number: 5
```

Sample Output

```
===== Mini Calculator =====
Enter first number: 17
Enter second number: 5

--- Results ---
Addition:        17.0 + 5.0 = 22.0
Subtraction:     17.0 - 5.0 = 12.0
Multiplication:  17.0 * 5.0 = 85.0
Division:        17.0 / 5.0 = 3.4
Floor Division:  17.0 // 5.0 = 3.0
Modulus:         17.0 % 5.0 = 2.0
Exponentiation:  17.0 ** 5.0 = 1419857.0
```

Explanation of Output:

1. Addition: $17 + 5 = 22$.
2. Subtraction: $17 - 5 = 12$.
3. Multiplication: $17 \times 5 = 85$.
4. Division: $17 \div 5 = 3.4$.
5. Floor Division: $\lfloor 17 \div 5 \rfloor = 3$ (discards the decimal part).
6. Modulus: $17 \mod 5 = 2$ (remainder after division).
7. Exponentiation: $17^5 = 1419857$.
8. The program also checks if the second number is zero before performing division to avoid errors.

1.4 Learning Outcomes

Upon completing this lab, students will be able to:

1. Use the `print()` function with `sep` and `end` parameters effectively.
2. Accept user input using the `input()` function and perform type conversion.
3. Write and evaluate arithmetic expressions using Python operators.
4. Understand operator precedence and its effect on expression evaluation.
5. Develop simple interactive programs that perform calculations.

2 Lab 2: Decision Constructs (if, elif, nested if)

2.1 Objective

To understand and implement decision-making constructs in Python including `if`, `if-else`, `if-elif-else`, and nested conditional statements to control the flow of program execution based on conditions.

2.2 Theory

Decision constructs (also called conditional statements or selection statements) are fundamental control structures in programming that allow a program to execute different blocks of code based on whether a condition evaluates to `True` or `False`. In Python, decision-making is implemented using `if`, `elif`, and `else` keywords.

The if Statement: The simplest form of decision-making. The code block under `if` is executed only when the condition is `True`. Python uses indentation (typically 4 spaces) to define code blocks, unlike languages that use curly braces.

The if-else Statement: When there are two possible paths of execution, `if-else` is used. If the condition is `True`, the `if` block executes; otherwise, the `else` block executes. This ensures that one of the two blocks always executes.

The if-elif-else Statement: When there are multiple conditions to check, the `elif` (short for "else if") keyword is used. Python evaluates conditions from top to bottom and executes the block corresponding to the first `True` condition. If none of the conditions are `True`, the `else` block executes (if present).

Nested if Statements: An `if` statement inside another `if` statement is called nested `if`. This is useful when a decision depends on the outcome of a previous decision. While nesting can be powerful, excessive nesting can make code harder to read and maintain.

Comparison Operators: Python provides six comparison operators: `==` (equal to), `!=` (not equal to), `>` (greater than), `<` (less than), `>=` (greater than or equal to), and `<=` (less than or equal to). These operators return Boolean values (`True` or `False`).

Logical Operators: Multiple conditions can be combined using logical operators: `and` (both conditions must be True), `or` (at least one condition must be True), and `not` (negates the condition).

2.3 Programs

2.3.1 Basic Programs

Program 1: Even or Odd

Problem Statement: Write a Python program to check whether a given number is even or odd.

Algorithm:

Step 1: Start the program.
Step 2: Accept an integer from the user.
Step 3: Check if the number is divisible by 2 (i.e., `num % 2 == 0`).
Step 4: If yes, print "Even"; otherwise, print "Odd".
Step 5: End the program.

```python
# Program 1: Even or Odd
num = int(input("Enter a number: "))

if num % 2 == 0:
    print(num, "is an Even number.")
else:
    print(num, "is an Odd number.")
```

Sample Input:

```
Enter a number: 24
```

Sample Output

```
Enter a number: 24
24 is an Even number.
```

Explanation of Output:

1. The user enters 24.
2. The modulus operation $24 \mod 2 = 0$, so the condition `num % 2 == 0` is `True`.
3. Since the condition is True, the `if` block executes, printing "24 is an Even number."
4. If the user had entered 25, $25 \mod 2 = 1 \neq 0$, and the `else` block would execute.

Program 2: Positive or Negative

Problem Statement: Write a Python program to check whether a number is positive, negative, or zero.

Algorithm:

Step 1: Start the program.
Step 2: Accept a number from the user.
Step 3: If the number is greater than 0, print "Positive".
Step 4: Else if the number is less than 0, print "Negative".
Step 5: Else, print "Zero".
Step 6: End the program.

```python
# Program 2: Positive or Negative
num = float(input("Enter a number: "))
```

```python
if num > 0:
    print(num, "is a Positive number.")
elif num < 0:
    print(num, "is a Negative number.")
else:
    print("The number is Zero.")
```

Sample Input:

```
Enter a number: -7.5
```

Sample Output

```
Enter a number: -7.5
-7.5 is a Negative number.
```

Explanation of Output:

1. The user enters -7.5.
2. The first condition `num > 0` is `False` (since $-7.5 < 0$).
3. The second condition `num < 0` is `True`.
4. Therefore, the `elif` block executes, printing "-7.5 is a Negative number."

Program 3: Voting Eligibility

Problem Statement: Write a Python program to check whether a person is eligible to vote based on their age.

Algorithm:

Step 1: Start the program.
Step 2: Accept the age from the user.
Step 3: If age is 18 or above, print "Eligible to vote".
Step 4: Otherwise, calculate years remaining and print "Not eligible".
Step 5: End the program.

```python
# Program 3: Voting eligibility
age = int(input("Enter your age: "))

if age >= 18:
    print("You are eligible to vote!")
else:
    years_left = 18 - age
    print("You are not eligible to vote.")
    print("You need to wait", years_left, "more year(s).")
```

Sample Input:

```
Enter your age: 15
```

Sample Output

```
Enter your age: 15
You are not eligible to vote.
You need to wait 3 more year(s).
```

Explanation of Output:

1. The user enters age 15.
2. The condition `age >= 18` evaluates to `False` (since $15 < 18$).
3. The `else` block executes, calculating $18 - 15 = 3$ years remaining.

Program 4: Divisible by 5

Problem Statement: Write a Python program to check whether a number is divisible by 5.

Algorithm:

Step 1: Start the program.
Step 2: Accept a number from the user.
Step 3: Check if the number modulo 5 equals 0.
Step 4: Display appropriate message.
Step 5: End the program.

```python
# Program 4: Divisible by 5
num = int(input("Enter a number: "))

if num % 5 == 0:
    print(num, "is divisible by 5.")
else:
    print(num, "is not divisible by 5.")
```

Sample Input:

`Enter a number: 35`

Sample Output

```
Enter a number: 35
35 is divisible by 5.
```

Explanation of Output:

1. The user enters 35.
2. $35 \mod 5 = 0$, so the condition is `True`.
3. The program prints "35 is divisible by 5."

Program 5: Largest of 2 Numbers

Problem Statement: Write a Python program to find the largest of two numbers.

Algorithm:

Step 1: Start the program.
Step 2: Accept two numbers from the user.
Step 3: Compare them using if-elif-else.
Step 4: Display the larger number or if they are equal.
Step 5: End the program.

```python
# Program 5: Largest of 2 numbers
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))

if a > b:
    print(a, "is the largest.")
elif b > a:
    print(b, "is the largest.")
else:
    print("Both numbers are equal.")
```

Sample Input:

```
Enter first number: 42
Enter second number: 37
```

Sample Output

```
Enter first number: 42
Enter second number: 37
42.0 is the largest.
```

Explanation of Output:

1. The user enters 42 and 37.
2. The condition `a > b` evaluates to `True` ($42 > 37$).
3. The program prints "42.0 is the largest."

2.3.2 Medium Programs

Program 6: Largest of 3 Numbers

Problem Statement: Write a Python program to find the largest of three numbers using nested if statements.

Algorithm:

Step 1: Start the program.
Step 2: Accept three numbers from the user.
Step 3: Compare the first number with the second and third.
Step 4: If the first is not the largest, compare the second with the third.
Step 5: Display the largest number.
Step 6: End the program.

```python
# Program 6: Largest of 3 numbers
a = float(input("Enter first number: "))
b = float(input("Enter second number: "))
c = float(input("Enter third number: "))

if a >= b and a >= c:
    largest = a
elif b >= a and b >= c:
    largest = b
else:
    largest = c

print("The largest number is:", largest)
```

Sample Input:

```
Enter first number: 45
Enter second number: 78
Enter third number: 56
```

Sample Output

```
Enter first number: 45
Enter second number: 78
Enter third number: 56
The largest number is: 78.0
```

Explanation of Output:

1. Three numbers are entered: 45, 78, and 56.
2. First condition: Is $45 \geq 78$ AND $45 \geq 56$? No ($45 < 78$).
3. Second condition: Is $78 \geq 45$ AND $78 \geq 56$? Yes. So `largest = 78`.
4. The program prints "The largest number is: 78.0".

Program 7: Grade Calculation

Problem Statement: Write a Python program to assign a grade based on the marks obtained by a student.

Algorithm:

Step 1: Start the program.
Step 2: Accept marks from the user (0–100).
Step 3: Use if-elif-else to determine the grade:
- ≥ 90: Grade A+
- ≥ 80: Grade A
- ≥ 70: Grade B
- ≥ 60: Grade C
- ≥ 50: Grade D
- < 50: Fail

Step 4: Display the grade.
Step 5: End the program.

```python
# Program 7: Grade calculation
marks = float(input("Enter your marks (0-100): "))

if marks < 0 or marks > 100:
    print("Invalid marks! Please enter between 0 and 100.")
elif marks >= 90:
    grade = "A+"
    print("Grade:", grade, "- Outstanding!")
elif marks >= 80:
    grade = "A"
    print("Grade:", grade, "- Excellent!")
elif marks >= 70:
    grade = "B"
    print("Grade:", grade, "- Good!")
elif marks >= 60:
    grade = "C"
    print("Grade:", grade, "- Average")
elif marks >= 50:
    grade = "D"
    print("Grade:", grade, "- Below Average")
else:
    grade = "F"
    print("Grade:", grade, "- Fail. Please improve.")
```

Sample Input:

```
Enter your marks (0-100): 85
```

Sample Output

```
Enter your marks (0-100): 85
Grade: A - Excellent!
```

Explanation of Output:

1. The user enters 85 marks.
2. The first condition checks validity (85 is between 0 and 100, so valid).

3. $85 \geq 90$? No. $85 \geq 80$? Yes. So grade is "A".
4. The program prints "Grade: A – Excellent!"

Program 8: Electricity Bill Calculation

Problem Statement: Write a Python program to calculate the electricity bill based on units consumed using a slab-based pricing system.

Algorithm:

Step 1: Start the program.
Step 2: Accept units consumed from the user.
Step 3: Calculate bill using slabs:
- First 100 units: Rs. 1.50 per unit
- Next 200 units (101–300): Rs. 2.50 per unit
- Next 200 units (301–500): Rs. 4.00 per unit
- Above 500 units: Rs. 6.00 per unit

Step 4: Display the total bill.
Step 5: End the program.

```python
# Program 8: Electricity bill calculation
units = int(input("Enter units consumed: "))

if units <= 100:
    bill = units * 1.50
elif units <= 300:
    bill = 100 * 1.50 + (units - 100) * 2.50
elif units <= 500:
    bill = 100 * 1.50 + 200 * 2.50 + (units - 300) * 4.00
else:
    bill = 100 * 1.50 + 200 * 2.50 + 200 * 4.00 + (units - 500) *
        6.00

print("Units Consumed:", units)
print("Total Bill: Rs.", round(bill, 2))
```

Sample Input:

```
Enter units consumed: 350
```

Sample Output

```
Enter units consumed: 350
Units Consumed: 350
Total Bill: Rs. 850.0
```

Explanation of Output:

1. Units consumed: 350 (falls in the third slab).
2. First 100 units: $100 \times 1.50 = 150$.

3. Next 200 units (101–300): $200 \times 2.50 = 500$.
4. Remaining 50 units (301–350): $50 \times 4.00 = 200$.
5. Total: $150 + 500 + 200 = 850$.

Program 9: Leap Year Check

Problem Statement: Write a Python program to check whether a given year is a leap year.

Algorithm:

Step 1: Start the program.
Step 2: Accept a year from the user.
Step 3: A year is a leap year if:
- It is divisible by 4 AND
- It is NOT divisible by 100, OR
- It is divisible by 400.

Step 4: Display the result.
Step 5: End the program.

```python
# Program 9: Leap year check
year = int(input("Enter a year: "))

if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(year, "is a Leap Year.")
else:
    print(year, "is NOT a Leap Year.")
```

Sample Input:

`Enter a year: 2024`

Sample Output

```
Enter a year: 2024
2024 is a Leap Year.
```

Explanation of Output:

1. The user enters 2024.
2. $2024 \mod 4 = 0$ (divisible by 4) and $2024 \mod 100 = 24 \neq 0$ (not divisible by 100).
3. Since both parts of the first condition are True, the overall condition is True.
4. The program prints "2024 is a Leap Year."

Note:

Century years (like 1900, 2100) are NOT leap years unless they are divisible by 400 (like 2000). This is why we need the second condition.

Program 10: Simple Calculator

Problem Statement: Write a Python program that acts as a simple calculator using decision constructs.

Algorithm:

Step 1: Start the program.
Step 2: Accept two numbers and an operator from the user.
Step 3: Use if-elif-else to perform the corresponding operation.
Step 4: Handle division by zero.
Step 5: Display the result.
Step 6: End the program.

```python
# Program 10: Simple calculator
print("===== Simple Calculator =====")
num1 = float(input("Enter first number: "))
operator = input("Enter operator (+, -, *, /): ")
num2 = float(input("Enter second number: "))

if operator == "+":
    result = num1 + num2
    print("Result:", num1, "+", num2, "=", result)
elif operator == "-":
    result = num1 - num2
    print("Result:", num1, "-", num2, "=", result)
elif operator == "*":
    result = num1 * num2
    print("Result:", num1, "*", num2, "=", result)
elif operator == "/":
    if num2 != 0:
        result = num1 / num2
        print("Result:", num1, "/", num2, "=", round(result, 4))
    else:
        print("Error: Division by zero is not allowed!")
else:
    print("Invalid operator! Please use +, -, *, or /")
```

Sample Input:

```
Enter first number: 15
Enter operator (+, -, *, /): *
Enter second number: 7
```

Sample Output

```
===== Simple Calculator =====
Enter first number: 15
Enter operator (+, -, *, /): *
Enter second number: 7
Result: 15.0 * 7.0 = 105.0
```

Explanation of Output:

1. The user enters 15, operator "*", and 7.
2. The program checks each condition: "+"? No. "-"? No. "*"? Yes.
3. Multiplication is performed: $15 \times 7 = 105$.
4. The result is displayed.

2.3.3 Advanced Programs

Program 11: Scholarship System Using Nested if

Problem Statement: Write a Python program that determines scholarship eligibility based on marks, family income, and attendance percentage using nested if statements.

Algorithm:

Step 1: Start the program.
Step 2: Accept marks, family income, and attendance from the user.
Step 3: Check if marks are above 80 (first level).
Step 4: If yes, check if family income is below Rs. 5,00,000 (second level).
Step 5: If yes, check if attendance is above 85% (third level).
Step 6: Assign scholarship type based on conditions met.
Step 7: Display the result.
Step 8: End the program.

```python
# Program 11: Scholarship system using nested if
print("===== Scholarship Eligibility System =====")
marks = float(input("Enter your marks (0-100): "))
income = float(input("Enter family annual income (Rs.): "))
attendance = float(input("Enter attendance percentage: "))

if marks >= 80:
    if income <= 500000:
        if attendance >= 85:
            print("\nCongratulations! You are eligible for FULL
                Scholarship!")
            print("Scholarship Type: Merit-cum-Means")
            print("Benefits: 100% tuition fee waiver")
        else:
            print("\nYou are eligible for PARTIAL Scholarship.")
            print("Scholarship Type: Merit-based")
            print("Note: Improve attendance above 85% for full
                scholarship.")
    else:
        if marks >= 90:
            print("\nYou are eligible for MERIT Scholarship!")
            print("Scholarship Type: Pure Merit")
            print("Benefits: 50% tuition fee waiver")
        else:
```

```
            print("\nScholarship not available.")
            print("Reason: Income exceeds limit and marks below
               90.")
else:
    if marks >= 60:
        print("\nNot eligible for scholarship.")
        print("Suggestion: Score above 80 marks for scholarship
           eligibility.")
    else:
        print("\nNot eligible for scholarship.")
        print("Suggestion: Focus on improving academic
           performance.")
```

Sample Input:

```
Enter your marks (0-100): 92
Enter family annual income (Rs.): 350000
Enter attendance percentage: 90
```

Sample Output

```
===== Scholarship Eligibility System =====
Enter your marks (0-100): 92
Enter family annual income (Rs.): 350000
Enter attendance percentage: 90

Congratulations! You are eligible for FULL Scholarship!
Scholarship Type: Merit-cum-Means
Benefits: 100% tuition fee waiver
```

Explanation of Output:

1. Marks: 92 (≥ 80, first condition True).
2. Income: Rs. 3,50,000 ($\leq 5,00,000$, second condition True).
3. Attendance: 90% ($\geq 85\%$, third condition True).
4. All three nested conditions are satisfied, so full scholarship is awarded.

Program 12: Menu-Driven Decision Program

Problem Statement: Write a menu-driven Python program that offers various mathematical operations and allows the user to choose an operation.

Algorithm:

Step 1: Start the program.
Step 2: Display a menu with options: Area of shapes, Number checks, Conversions.
Step 3: Accept the user's choice.
Step 4: Based on the choice, ask for relevant inputs.
Step 5: Perform the selected operation.
Step 6: Display the result.
Step 7: End the program.

```python
# Program 12: Menu-driven decision program
print("=" * 40)
print("     MATH OPERATIONS MENU")
print("=" * 40)
print("1. Area of Rectangle")
print("2. Area of Triangle")
print("3. Check Prime (Simple)")
print("4. Celsius to Fahrenheit")
print("5. Check Palindrome Number")
print("=" * 40)

choice = int(input("Enter your choice (1-5): "))

if choice == 1:
    length = float(input("Enter length: "))
    width = float(input("Enter width: "))
    area = length * width
    print("Area of Rectangle:", area)

elif choice == 2:
    base = float(input("Enter base: "))
    height = float(input("Enter height: "))
    area = 0.5 * base * height
    print("Area of Triangle:", area)

elif choice == 3:
    num = int(input("Enter a number: "))
    if num <= 1:
        print(num, "is not a Prime number.")
    elif num <= 3:
        print(num, "is a Prime number.")
    elif num % 2 == 0 or num % 3 == 0:
        print(num, "is not a Prime number.")
    else:
        print(num, "might be Prime (basic check).")

elif choice == 4:
    celsius = float(input("Enter temperature in Celsius: "))
    fahrenheit = (9/5) * celsius + 32
    print(celsius, "^^b0C =", round(fahrenheit, 2), "^^b0F")
```

```python
elif choice == 5:
    num = int(input("Enter a number: "))
    original = num
    reverse = 0
    temp = num
    while temp > 0:
        digit = temp % 10
        reverse = reverse * 10 + digit
        temp //= 10
    if original == reverse:
        print(original, "is a Palindrome number.")
    else:
        print(original, "is not a Palindrome number.")

else:
    print("Invalid choice! Please select 1-5.")
```

Sample Input:

```
Enter your choice (1-5): 2
Enter base: 10
Enter height: 5
```

Sample Output

```
========================================
     MATH OPERATIONS MENU
========================================
1. Area of Rectangle
2. Area of Triangle
3. Check Prime (Simple)
4. Celsius to Fahrenheit
5. Check Palindrome Number
========================================
Enter your choice (1-5): 2
Enter base: 10
Enter height: 5
Area of Triangle: 25.0
```

Explanation of Output:

1. The menu displays 5 options and the user selects option 2 (Area of Triangle).
2. The program asks for base (10) and height (5).
3. Area is calculated: $\frac{1}{2} \times 10 \times 5 = 25.0$.
4. The result is displayed.

2.4 Learning Outcomes

Upon completing this lab, students will be able to:

1. Implement simple `if-else` conditions for binary decisions.
2. Use `if-elif-else` chains for multi-way branching.
3. Apply nested `if` statements for complex decision trees.
4. Combine logical operators with comparison operators.
5. Design menu-driven programs using decision constructs.

3 Lab 3 & 4: Loops (for, while, break, continue, pass)

3.1 Objective

To understand and implement iterative constructs in Python including `for` loops, `while` loops, and loop control statements (`break`, `continue`, `pass`) to perform repetitive tasks efficiently.

3.2 Theory

Loops are control structures that allow a block of code to be executed repeatedly. Python provides two main types of loops: `for` and `while`.

The for Loop: The `for` loop in Python is used to iterate over a sequence (such as a list, tuple, string, or range). Unlike C/Java where `for` loops use counter variables, Python's `for` loop is more like a "for-each" loop. The `range()` function is commonly used to generate a sequence of numbers. `range(n)` generates numbers from 0 to $n-1$; `range(start, stop)` generates from start to $stop-1$; and `range(start, stop, step)` allows specifying the increment.

The while Loop: The `while` loop executes a block of code as long as a given condition remains `True`. It is used when the number of iterations is not known beforehand. Care must be taken to ensure the loop condition eventually becomes `False` to avoid infinite loops.

Loop Control Statements:

- **break:** Terminates the loop entirely and transfers control to the statement immediately after the loop.
- **continue:** Skips the remaining code in the current iteration and moves to the next iteration.
- **pass:** A null statement that does nothing. It is used as a placeholder when a statement is syntactically required but no action is needed.

Nested Loops: A loop inside another loop is called a nested loop. The inner loop completes all its iterations for each single iteration of the outer loop. Nested loops are commonly used for printing patterns, working with matrices, and solving problems that require examining all pairs of elements.

The `else` clause with loops in Python is a unique feature: the `else` block after a loop executes only if the loop completes normally (without encountering a `break` statement).

3.3 Programs

3.3.1 Basic Programs

Program 1: Print Numbers 1 to N

Problem Statement: Write a Python program to print all numbers from 1 to N, where N is entered by the user.

Algorithm:

Step 1: Start the program.
Step 2: Accept N from the user.
Step 3: Use a `for` loop with `range(1, N+1)`.
Step 4: Print each number.
Step 5: End the program.

```python
# Program 1: Print numbers 1 to N
n = int(input("Enter N: "))
print("Numbers from 1 to", n, ":")
for i in range(1, n + 1):
    print(i, end=" ")
print()  # New line at end
```

Sample Input:

```
Enter N: 10
```

Sample Output

```
Enter N: 10
Numbers from 1 to 10 :
1 2 3 4 5 6 7 8 9 10
```

Explanation of Output:

1. The user enters $N = 10$.
2. `range(1, 11)` generates numbers 1, 2, 3, ..., 10.
3. Each number is printed with a space separator using `end=‘‘ ’’`.
4. The final `print()` adds a newline after the sequence.

Program 2: Sum of N Numbers

Problem Statement: Write a Python program to calculate the sum of the first N natural numbers.

Algorithm:

Step 1: Start the program.
Step 2: Accept N from the user.
Step 3: Initialize sum to 0.
Step 4: Use a loop to add each number from 1 to N to the sum.
Step 5: Display the total sum.
Step 6: End the program.

```python
# Program 2: Sum of N numbers
n = int(input("Enter N: "))
total = 0
```

```python
for i in range(1, n + 1):
    total += i

print("Sum of first", n, "natural numbers:", total)

# Verification using formula
formula_result = n * (n + 1) // 2
print("Verification (using formula n*(n+1)/2):", formula_result)
```

Sample Input:

```
Enter N: 100
```

Sample Output

```
Enter N: 100
Sum of first 100 natural numbers: 5050
Verification (using formula n*(n+1)/2): 5050
```

Explanation of Output:

1. The loop runs from 1 to 100, accumulating the sum: $1 + 2 + 3 + ... + 100 = 5050$.
2. This is verified using the formula $\frac{n(n+1)}{2} = \frac{100 \times 101}{2} = 5050$.

Program 3: Multiplication Table

Problem Statement: Write a Python program to display the multiplication table of a given number.

Algorithm:

Step 1: Start the program.
Step 2: Accept a number from the user.
Step 3: Use a `for` loop from 1 to 10.
Step 4: For each iteration, multiply the number by the loop variable.
Step 5: Display the result in a formatted manner.
Step 6: End the program.

```python
# Program 3: Multiplication table
num = int(input("Enter a number: "))
print("\nMultiplication Table of", num)
print("-" * 25)
for i in range(1, 11):
    print(num, "x", i, "=", num * i)
```

Sample Input:

```
Enter a number: 7
```

Sample Output

```
Enter a number: 7

Multiplication Table of 7
-------------------------
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Explanation of Output:

1. The loop variable `i` goes from 1 to 10.
2. For each value of `i`, the product $7 \times i$ is computed and displayed.
3. The output is formatted to show the multiplication expression and result.

Program 4: Count Digits in a Number

Problem Statement: Write a Python program to count the number of digits in a given integer using a while loop.

Algorithm:

Step 1: Start the program.
Step 2: Accept a number from the user.
Step 3: Handle negative numbers by taking absolute value.
Step 4: Initialize a counter to 0.
Step 5: Repeatedly divide the number by 10 until it becomes 0.
Step 6: Increment the counter in each iteration.
Step 7: Display the count.
Step 8: End the program.

```python
# Program 4: Count digits in a number
num = int(input("Enter a number: "))
original = num
num = abs(num)  # Handle negative numbers
count = 0

if num == 0:
    count = 1
else:
    while num > 0:
        num //= 10
        count += 1
```

```
print("Number of digits in", original, "is:", count)
```

Sample Input:

```
Enter a number: 54321
```

Sample Output

```
Enter a number: 54321
Number of digits in 54321 is: 5
```

Explanation of Output:

1. The number 54321 is divided by 10 repeatedly: $54321 \to 5432 \to 543 \to 54 \to 5 \to 0$.
2. The loop runs 5 times before the number becomes 0.
3. Therefore, the digit count is 5.

Program 5: Reverse a Number

Problem Statement: Write a Python program to reverse the digits of a given number.

Algorithm:

Step 1: Start the program.
Step 2: Accept a number from the user.
Step 3: Initialize `reverse` to 0.
Step 4: While the number is greater than 0:
- Extract the last digit using modulus.
- Append it to `reverse` by multiplying reverse by 10 and adding the digit.
- Remove the last digit using floor division.

Step 5: Display the reversed number.
Step 6: End the program.

```
# Program 5: Reverse a number
num = int(input("Enter a number: "))
original = num
reverse = 0

while num > 0:
    digit = num % 10          # Extract last digit
    reverse = reverse * 10 + digit   # Build reversed number
    num //= 10                # Remove last digit

print("Original number:", original)
print("Reversed number:", reverse)
```

Sample Input:

```
Enter a number: 12345
```

Sample Output

```
Enter a number: 12345
Original number: 12345
Reversed number: 54321
```

Explanation of Output:

1. Iteration 1: digit $= 12345 \mod 10 = 5$, reverse $= 0 \times 10 + 5 = 5$, num $= 1234$.
2. Iteration 2: digit $= 1234 \mod 10 = 4$, reverse $= 5 \times 10 + 4 = 54$, num $= 123$.
3. Iteration 3: digit $= 3$, reverse $= 543$, num $= 12$.
4. Iteration 4: digit $= 2$, reverse $= 5432$, num $= 1$.
5. Iteration 5: digit $= 1$, reverse $= 54321$, num $= 0$. Loop ends.

3.3.2 Medium Programs

Program 6: Prime Number Check

Problem Statement: Write a Python program to check whether a given number is a prime number.

Algorithm:

Step 1: Start the program.
Step 2: Accept a number from the user.
Step 3: If the number is less than 2, it is not prime.
Step 4: Check divisibility from 2 to $\sqrt{n}$.
Step 5: If any divisor is found, the number is not prime.
Step 6: Otherwise, the number is prime.
Step 7: End the program.

```python
# Program 6: Prime number check
num = int(input("Enter a number: "))

if num < 2:
    print(num, "is not a Prime number.")
else:
    is_prime = True
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            is_prime = False
            break

    if is_prime:
        print(num, "is a Prime number.")
    else:
        print(num, "is not a Prime number.")
```

Sample Input:

```
Enter a number: 29
```

Sample Output

```
Enter a number: 29
29 is a Prime number.
```

Explanation of Output:

1. The number 29 is greater than 1, so we check for factors.
2. $\sqrt{29} \approx 5.38$, so we check divisibility by 2, 3, 4, 5.
3. $29 \mod 2 = 1$, $29 \mod 3 = 2$, $29 \mod 4 = 1$, $29 \mod 5 = 4$. None divide evenly.
4. Since no factor is found, 29 is a prime number.

Note:

We only need to check up to $\sqrt{n}$ because if $n = a \times b$, then at least one of a or b must be $\leq \sqrt{n}$. This optimization significantly reduces the number of checks needed.

Program 7: Factorial Using Loop

Problem Statement: Write a Python program to compute the factorial of a number using a loop.

Algorithm:

Step 1: Start the program.
Step 2: Accept a non-negative integer N from the user.
Step 3: Initialize `factorial` to 1.
Step 4: Multiply `factorial` by each number from 1 to N.
Step 5: Display the result.
Step 6: End the program.

```python
# Program 7: Factorial using loop
num = int(input("Enter a non-negative integer: "))

if num < 0:
    print("Factorial is not defined for negative numbers.")
elif num == 0:
    print("Factorial of 0 is: 1")
else:
    factorial = 1
    for i in range(1, num + 1):
        factorial *= i
    print("Factorial of", num, "is:", factorial)
```

Sample Input:

```
Enter a non-negative integer: 6
```

Sample Output

```
Enter a non-negative integer: 6
Factorial of 6 is: 720
```

Explanation of Output:

1. $6! = 1 \times 2 \times 3 \times 4 \times 5 \times 6 = 720$.
2. The loop multiplies: $1 \to 2 \to 6 \to 24 \to 120 \to 720$.

Program 8: Fibonacci Series

Problem Statement: Write a Python program to generate the Fibonacci series up to N terms.

Algorithm:

Step 1: Start the program.
Step 2: Accept N (number of terms) from the user.
Step 3: Initialize first two terms: $a = 0$, $b = 1$.
Step 4: Print the first term.
Step 5: Use a loop to generate subsequent terms by adding the previous two.
Step 6: End the program.

```python
# Program 8: Fibonacci series
n = int(input("Enter number of terms: "))

a, b = 0, 1
print("Fibonacci Series up to", n, "terms:")

for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
print()
```

Sample Input:

`Enter number of terms: 10`

Sample Output

```
Enter number of terms: 10
Fibonacci Series up to 10 terms:
0 1 1 2 3 5 8 13 21 34
```

Explanation of Output:

1. Starting values: $a = 0$, $b = 1$.
2. Each new term is the sum of the previous two: $0, 1, 1, 2, 3, 5, 8, 13, 21, 34$.
3. The tuple assignment `a, b = b, a + b` simultaneously updates both variables.

Program 9: Palindrome Number

Problem Statement: Write a Python program to check whether a number is a palindrome.

Algorithm:

Step 1: Start the program.
Step 2: Accept a number from the user.
Step 3: Reverse the number using a while loop.
Step 4: Compare the reversed number with the original.
Step 5: If they are equal, it is a palindrome.
Step 6: End the program.

```python
# Program 9: Palindrome number
num = int(input("Enter a number: "))
original = num
reverse = 0

while num > 0:
    digit = num % 10
    reverse = reverse * 10 + digit
    num //= 10

if original == reverse:
    print(original, "is a Palindrome number.")
else:
    print(original, "is NOT a Palindrome number.")
```

Sample Input:

```
Enter a number: 12321
```

Sample Output

```
Enter a number: 12321
12321 is a Palindrome number.
```

Explanation of Output:

1. The number 12321 is reversed to get 12321.
2. Since the original (12321) equals the reversed (12321), it is a palindrome.
3. A palindrome reads the same forwards and backwards.

Program 10: Pattern Printing (Stars)

Problem Statement: Write a Python program to print a right-angled triangle pattern using stars.

Algorithm:

Step 1: Start the program.
Step 2: Accept the number of rows from the user.
Step 3: Use a nested loop: outer loop for rows, inner loop for columns.
Step 4: In each row, print stars equal to the row number.
Step 5: End the program.

```python
# Program 10: Pattern printing (stars)
n = int(input("Enter number of rows: "))

print("\nRight-angled Triangle Pattern:")
for i in range(1, n + 1):
    for j in range(1, i + 1):
        print("*", end=" ")
    print()  # Move to next line
```

Sample Input:

```
Enter number of rows: 5
```

Sample Output

```
Enter number of rows: 5

Right-angled Triangle Pattern:
*
* *
* * *
* * * *
* * * * *
```

Explanation of Output:

1. Row 1: inner loop runs 1 time → 1 star.
2. Row 2: inner loop runs 2 times → 2 stars.
3. Row 3: 3 stars, Row 4: 4 stars, Row 5: 5 stars.
4. The `print()` after the inner loop moves to the next line.

3.3.3 Advanced Programs

Program 11: Pyramid Pattern Using Nested Loops

Problem Statement: Write a Python program to print a centered pyramid pattern and an inverted pyramid.

Algorithm:

Step 1: Start the program.
Step 2: Accept the number of rows from the user.
Step 3: For the pyramid: print leading spaces followed by stars.
Step 4: For the inverted pyramid: reverse the process.
Step 5: End the program.

```python
# Program 11: Pyramid pattern using nested loops
n = int(input("Enter number of rows: "))

# Upright Pyramid
print("\n--- Pyramid Pattern ---")
for i in range(1, n + 1):
    # Print spaces
    print(" " * (n - i), end="")
    # Print stars
    print("* " * i)

# Inverted Pyramid
print("\n--- Inverted Pyramid ---")
for i in range(n, 0, -1):
    print(" " * (n - i), end="")
    print("* " * i)

# Diamond Pattern
print("\n--- Diamond Pattern ---")
for i in range(1, n + 1):
    print(" " * (n - i), end="")
    print("* " * i)
for i in range(n - 1, 0, -1):
    print(" " * (n - i), end="")
    print("* " * i)
```

Sample Input:

```
Enter number of rows: 4
```

Sample Output

```
Enter number of rows: 4

--- Pyramid Pattern ---
   *
  * *
 * * *
* * * *

--- Inverted Pyramid ---
* * * *
 * * *
```

```
  * *
   *

--- Diamond Pattern ---
   *
  * *
 * * *
* * * *
 * * *
  * *
   *
```

Explanation of Output:

1. **Pyramid:** Row i has $(n - i)$ leading spaces and i stars. This creates a centered triangle.
2. **Inverted Pyramid:** The loop runs from n down to 1, reversing the pattern.
3. **Diamond:** The upper half is a pyramid (rows 1 to n), and the lower half is an inverted pyramid (rows $n - 1$ to 1).

Program 12: Menu-Driven Loop Program

Problem Statement: Write a menu-driven program that demonstrates different loop operations and continues until the user chooses to exit.

Algorithm:

Step 1: Start the program.
Step 2: Display a menu with loop-based operations.
Step 3: Accept the user's choice.
Step 4: Perform the selected operation.
Step 5: Ask if the user wants to continue.
Step 6: Repeat until the user chooses to exit.
Step 7: End the program.

```python
# Program 12: Menu-driven loop program
while True:
    print("\n" + "=" * 35)
    print("   LOOP OPERATIONS MENU")
    print("=" * 35)
    print("1. Sum of Even numbers (1 to N)")
    print("2. Sum of Odd numbers (1 to N)")
    print("3. Power calculation (x^n)")
    print("4. GCD of two numbers")
    print("5. Exit")
    print("=" * 35)

    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
        n = int(input("Enter N: "))
        total = 0
        for i in range(2, n + 1, 2):
            total += i
        print("Sum of even numbers from 1 to", n, ":", total)

    elif choice == 2:
        n = int(input("Enter N: "))
        total = 0
        for i in range(1, n + 1, 2):
            total += i
        print("Sum of odd numbers from 1 to", n, ":", total)

    elif choice == 3:
        x = int(input("Enter base (x): "))
        n = int(input("Enter exponent (n): "))
        result = 1
        for i in range(n):
            result *= x
        print(x, "^", n, "=", result)

    elif choice == 4:
        a = int(input("Enter first number: "))
        b = int(input("Enter second number: "))
        while b != 0:
            a, b = b, a % b
        print("GCD:", a)

    elif choice == 5:
        print("Thank you! Exiting program.")
        break

    else:
        print("Invalid choice! Try again.")
```

Sample Input:

```
Enter your choice: 4
Enter first number: 36
Enter second number: 24
```

Sample Output

```
===================================
   LOOP OPERATIONS MENU
===================================
1. Sum of Even numbers (1 to N)
2. Sum of Odd numbers (1 to N)
3. Power calculation (x^n)
4. GCD of two numbers
5. Exit
===================================
```

```
Enter your choice: 4
Enter first number: 36
Enter second number: 24
GCD: 12
```

Explanation of Output:

1. The user selects option 4 (GCD calculation).
2. GCD is computed using Euclid's algorithm:
   - $a = 36, b = 24$: $a, b = 24, 36\%24 = 24, 12$
   - $a = 24, b = 12$: $a, b = 12, 24\%12 = 12, 0$
   - $b = 0$, so GCD is $a = 12$.
3. The `while True` loop keeps the menu running until the user selects Exit (option 5), which triggers `break`.

3.4 Learning Outcomes

Upon completing this lab, students will be able to:

1. Implement `for` loops with `range()` for counting iterations.
2. Use `while` loops for condition-based repetition.
3. Apply `break`, `continue`, and `pass` statements effectively.
4. Create nested loops for pattern printing and matrix operations.
5. Design menu-driven programs using loops.

4 Lab 5: Strings, Functions, and Recursion

4.1 Objective

To understand string manipulation techniques, function definition and invocation, parameter passing mechanisms, and recursive problem solving in Python.

4.2 Theory

Strings: Strings in Python are sequences of characters enclosed in single quotes (`' '`), double quotes (`" "`), or triple quotes (`''' '''` or `""" """`). Strings are **immutable**, meaning once created, their contents cannot be changed. Common string operations include: concatenation (`+`), repetition (`*`), indexing (`[i]`), slicing (`[start:end:step]`), and various built-in methods like `upper()`, `lower()`, `strip()`, `split()`, `join()`, `find()`, `replace()`, and `count()`.

Functions: A function is a reusable block of code that performs a specific task. Functions promote code reusability, modularity, and readability. In Python, functions are defined using the `def` keyword followed by the function name and parentheses containing optional parameters. Functions can have default parameter values, accept variable-length arguments (`*args`, `**kwargs`), and return values using the `return` statement.

Recursion: Recursion is a programming technique where a function calls itself to solve a problem. Every recursive function must have a **base case** (a condition that stops the recursion) and a **recursive case** (where the function calls itself with a modified argument). Recursion is particularly elegant for problems that have a natural recursive structure, such as factorial computation, Fibonacci sequence generation, and tree traversals. However, recursion can be memory-intensive due to the call stack, and excessive recursion depth can cause a `RecursionError`.

Python has a default recursion limit of 1000 calls, which can be modified using `sys.setrecursionlimit()`, though this should be done cautiously.

4.3 Programs

4.3.1 Basic Programs

Program 1: Reverse a String

Problem Statement: Write a Python program to reverse a given string.

Algorithm:

Step 1: Start the program.
Step 2: Accept a string from the user.
Step 3: Reverse the string using slicing `[::-1]`.
Step 4: Display the original and reversed strings.
Step 5: End the program.

```
# Program 1: Reverse a string
```

```python
text = input("Enter a string: ")
reversed_text = text[::-1]
print("Original string:", text)
print("Reversed string:", reversed_text)
```

Sample Input:

`Enter a string: CMR University`

Sample Output

```
Enter a string: CMR University
Original string: CMR University
Reversed string: ytisrevinU RMC
```

Explanation of Output:

1. The slicing notation `[::-1]` creates a reversed copy of the string.
2. `[::-1]` means: start from the end, go to the beginning, step by -1.
3. "CMR University" reversed character by character becomes "ytisrevinU RMC".

Program 2: Count Vowels

Problem Statement: Write a Python program to count the number of vowels in a given string.

```python
# Program 2: Count vowels
text = input("Enter a string: ")
vowels = "aeiouAEIOU"
count = 0

for char in text:
    if char in vowels:
        count += 1

print("String:", text)
print("Number of vowels:", count)
```

Sample Input:

`Enter a string: Python Programming`

Sample Output

```
Enter a string: Python Programming
String: Python Programming
Number of vowels: 4
```

Explanation of Output:

1. The program checks each character against the vowel string "aeiouAEIOU".
2. In "Python Programming": **o** (Python), **o** (Pr**o**gramming), **a** (Progr**a**mming), **i** (Programm**i**ng) = 4 vowels.

Program 3: Convert to Upper/Lower Case

```python
# Program 3: Convert to upper/lower case
text = input("Enter a string: ")

print("Original:  ", text)
print("Uppercase: ", text.upper())
print("Lowercase: ", text.lower())
print("Title Case:", text.title())
print("Swapcase:  ", text.swapcase())
```

Sample Input:

`Enter a string: Hello World`

Sample Output

```
Enter a string: Hello World
Original:   Hello World
Uppercase:  HELLO WORLD
Lowercase:  hello world
Title Case: Hello World
Swapcase:   hELLO wORLD
```

Explanation of Output:

1. `upper()` converts all characters to uppercase.
2. `lower()` converts all characters to lowercase.
3. `title()` capitalizes the first letter of each word.
4. `swapcase()` swaps the case of each character.

Program 4: Simple Function Example

```python
# Program 4: Simple function example
def greet(name):
    """This function greets the person passed in as a parameter
    """
    print("Hello,", name + "! Welcome to CMR University.")

def greet_with_time(name, time_of_day="morning"):
    """Greets with time of day"""
    print("Good", time_of_day + ",", name + "!")

# Function calls
greet("Alice")
greet("Bob")
greet_with_time("Charlie")
greet_with_time("Diana", "evening")
```

Sample Input: None

Sample Output

```
Hello, Alice! Welcome to CMR University.
Hello, Bob! Welcome to CMR University.
Good morning, Charlie!
Good evening, Diana!
```

Explanation of Output:

1. `greet("Alice")` passes "Alice" as the `name` parameter.
2. `greet_with_time("Charlie")` uses the default value "morning" for `time_of_day`.
3. `greet_with_time("Diana", "evening")` overrides the default with "evening".

Program 5: Even/Odd Using Function

```python
# Program 5: Even/Odd using function
def check_even_odd(num):
    """Returns 'Even' if num is even, 'Odd' otherwise"""
    if num % 2 == 0:
        return "Even"
    else:
        return "Odd"

# Main program
number = int(input("Enter a number: "))
result = check_even_odd(number)
print(number, "is", result)

# Test with multiple numbers
print("\nTesting multiple numbers:")
for n in [10, 15, 22, 33, 0]:
    print(n, "->", check_even_odd(n))
```

Sample Input:

`Enter a number: 17`

Sample Output

```
Enter a number: 17
17 is Odd

Testing multiple numbers:
10 -> Even
15 -> Odd
22 -> Even
33 -> Odd
0 -> Even
```

Explanation of Output:

1. The function `check_even_odd(17)` computes $17\%2 = 1 \neq 0$, so it returns "Odd".
2. The function is reused for multiple numbers, demonstrating **code reusability**.

3. Note that 0 is considered even since $0\%2 = 0$.

4.3.2 Medium Programs

Program 6: Palindrome String

```python
# Program 6: Palindrome string
def is_palindrome(text):
    """Check if a string is a palindrome (case-insensitive)"""
    cleaned = text.lower().replace(" ", "")
    return cleaned == cleaned[::-1]

# Main program
string = input("Enter a string: ")

if is_palindrome(string):
    print("'" + string + "' is a Palindrome.")
else:
    print("'" + string + "' is NOT a Palindrome.")

# Test with examples
print("\nMore examples:")
test_words = ["madam", "racecar", "hello", "level", "A man a plan
    a canal Panama"]
for word in test_words:
    status = "Palindrome" if is_palindrome(word) else "Not
       Palindrome"
    print(f"  '{word}' -> {status}")
```

Sample Input:

Enter a string: Racecar

Sample Output

```
Enter a string: Racecar
'Racecar' is a Palindrome.

More examples:
  'madam' -> Palindrome
  'racecar' -> Palindrome
  'hello' -> Not Palindrome
  'level' -> Palindrome
  'A man a plan a canal Panama' -> Palindrome
```

Program 7: Character Frequency

```python
# Program 7: Character frequency
def char_frequency(text):
    """Count frequency of each character in a string"""
    freq = {}
    for char in text:
        if char != ' ':  # Skip spaces
```

```
            freq[char] = freq.get(char, 0) + 1
    return freq

# Main program
string = input("Enter a string: ")
frequency = char_frequency(string)

print("\nCharacter Frequencies:")
for char, count in sorted(frequency.items()):
    print(f"  '{char}' : {count}")
print("Total unique characters:", len(frequency))
```

Sample Input:

Enter a string: banana

Sample Output

```
Enter a string: banana

Character Frequencies:
  'a' : 3
  'b' : 1
  'n' : 2
Total unique characters: 3
```

Program 8: Factorial Using Function

```
# Program 8: Factorial using function
def factorial(n):
    """Calculate factorial of n using iteration"""
    if n < 0:
        return "Undefined for negative numbers"
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

# Main program
num = int(input("Enter a number: "))
print(f"Factorial of {num} is: {factorial(num)}")

# Display factorials of 0 to 10
print("\nFactorials Table:")
for i in range(11):
    print(f"  {i}! = {factorial(i)}")
```

Sample Input:

Enter a number: 7

Sample Output

```
Enter a number: 7
Factorial of 7 is: 5040

Factorials Table:
  0! = 1
  1! = 1
  2! = 2
  3! = 6
  4! = 24
  5! = 120
  6! = 720
  7! = 5040
  8! = 40320
  9! = 362880
  10! = 3628800
```

Program 9: Fibonacci Using Function

```python
# Program 9: Fibonacci using function
def fibonacci(n):
    """Generate Fibonacci series of n terms"""
    if n <= 0:
        return []
    elif n == 1:
        return [0]

    series = [0, 1]
    for i in range(2, n):
        next_term = series[i-1] + series[i-2]
        series.append(next_term)
    return series

# Main program
terms = int(input("Enter number of terms: "))
fib_series = fibonacci(terms)
print(f"Fibonacci series ({terms} terms):")
print(fib_series)
```

Sample Input:

Enter number of terms: 12

Sample Output

```
Enter number of terms: 12
Fibonacci series (12 terms):
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Program 10: Sum of Digits Using Function

```python
# Program 10: Sum of digits using function
```

```python
def sum_of_digits(n):
    """Calculate sum of digits of a number"""
    n = abs(n)
    total = 0
    while n > 0:
        total += n % 10
        n //= 10
    return total

# Main program
num = int(input("Enter a number: "))
result = sum_of_digits(num)
print(f"Sum of digits of {num}: {result}")

# Show step-by-step
print("\nStep-by-step breakdown:")
temp = abs(num)
digits = []
while temp > 0:
    digits.append(temp % 10)
    temp //= 10
digits.reverse()
print(" + ".join(map(str, digits)), "=", result)
```

Sample Input:

```
Enter a number: 9876
```

Sample Output

```
Enter a number: 9876
Sum of digits of 9876: 30

Step-by-step breakdown:
9 + 8 + 7 + 6 = 30
```

4.3.3 Advanced Programs

Program 11: Factorial Using Recursion

```python
# Program 11: Factorial using recursion
def factorial_recursive(n, depth=0):
    """Calculate factorial recursively with trace"""
    indent = "  " * depth
    print(f"{indent}factorial({n}) called")

    # Base case
    if n == 0 or n == 1:
        print(f"{indent}factorial({n}) returns 1")
        return 1

    # Recursive case
    result = n * factorial_recursive(n - 1, depth + 1)
```

```python
    print(f"{indent}factorial({n}) returns {result}")
    return result

# Main program
num = int(input("Enter a number: "))
print("\nRecursion Trace:")
print("-" * 40)
result = factorial_recursive(num)
print("-" * 40)
print(f"\nFactorial of {num} = {result}")
```

Sample Input:

```
Enter a number: 5
```

Sample Output

```
Enter a number: 5

Recursion Trace:
----------------------------------------
factorial(5) called
  factorial(4) called
    factorial(3) called
      factorial(2) called
        factorial(1) called
        factorial(1) returns 1
      factorial(2) returns 2
    factorial(3) returns 6
  factorial(4) returns 24
factorial(5) returns 120
----------------------------------------

Factorial of 5 = 120
```

Note:

Every recursive function **must** have a base case to prevent infinite recursion. Without it, the program will crash with a `RecursionError`.

Program 12: Fibonacci Using Recursion

```python
# Program 12: Fibonacci using recursion
def fib(n):
    """Calculate nth Fibonacci number recursively"""
    if n <= 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n - 1) + fib(n - 2)

# Main program
```

```python
terms = int(input("Enter number of terms: "))
print(f"\nFibonacci Series ({terms} terms):")
for i in range(terms):
    print(fib(i), end=" ")
print()

# Show how recursion works for fib(5)
print("\nHow fib(5) is computed:")
print("fib(5) = fib(4) + fib(3)")
print("       = (fib(3) + fib(2)) + (fib(2) + fib(1))")
print("       = ((fib(2)+fib(1)) + (fib(1)+fib(0)))")
print("         + ((fib(1)+fib(0)) + 1)")
print("       = ((1+1) + (1+0)) + ((1+0) + 1)")
print("       = (2 + 1) + (1 + 1)")
print("       = 3 + 2 = 5")
```

Sample Input:

```
Enter number of terms: 10
```

Sample Output

```
Enter number of terms: 10

Fibonacci Series (10 terms):
0 1 1 2 3 5 8 13 21 34

How fib(5) is computed:
fib(5) = fib(4) + fib(3)
       = (fib(3) + fib(2)) + (fib(2) + fib(1))
       = ((fib(2)+fib(1)) + (fib(1)+fib(0)))
         + ((fib(1)+fib(0)) + 1)
       = ((1+1) + (1+0)) + ((1+0) + 1)
       = (2 + 1) + (1 + 1)
       = 3 + 2 = 5
```

Note:

The recursive Fibonacci has exponential time complexity $O(2^n)$. For large values, use **memoization** or **iterative** approach. Python's `functools.lru_cache` can optimize recursive functions.

4.4 Learning Outcomes

Upon completing this lab, students will be able to:

1. Perform string manipulations using built-in methods.
2. Define and call functions with parameters and return values.
3. Use default parameters and understand scope of variables.
4. Implement recursive solutions with proper base cases.
5. Compare iterative and recursive approaches.

5 Lab 6: Lists and Tuples (Mutable vs Immutable)

5.1 Objective

To understand and implement list and tuple data structures in Python, including their creation, manipulation, and the fundamental difference between mutable and immutable types.

5.2 Theory

Lists: A list is an ordered, mutable collection of items in Python. Lists are defined using square brackets `[ ]` and can contain elements of different data types. Being **mutable**, list elements can be changed, added, or removed after creation.

Tuples: A tuple is an ordered, **immutable** collection of items defined using parentheses `( )`. Once created, tuple elements cannot be modified. Tuples are faster than lists and can be used as dictionary keys.

List Comprehension: List comprehension provides a concise way to create lists: `[expression for item in iterable if condition]`.

5.3 Programs

5.3.1 Basic Programs

Program 1: Create a List

```python
# Program 1: Create a list
numbers = [10, 20, 30, 40, 50]
names = ["Alice", "Bob", "Charlie"]
mixed = [1, "hello", 3.14, True]

print("Numbers:", numbers)
print("Names:", names)
print("Mixed:", mixed)
print("\nType of numbers:", type(numbers))
print("Length:", len(numbers))
print("First element:", numbers[0])
print("Last element:", numbers[-1])
print("Slice [1:4]:", numbers[1:4])
```

Sample Output

```
Numbers: [10, 20, 30, 40, 50]
Names: ['Alice', 'Bob', 'Charlie']
Mixed: [1, 'hello', 3.14, True]

Type of numbers: <class 'list'>
Length: 5
First element: 10
Last element: 50
Slice [1:4]: [20, 30, 40]
```

Program 2: Find Max/Min

```python
# Program 2: Find max/min
n = int(input("How many numbers? "))
numbers = []

for i in range(n):
    num = int(input(f"Enter number {i+1}: "))
    numbers.append(num)

print("\nList:", numbers)
print("Maximum:", max(numbers))
print("Minimum:", min(numbers))
print("Sum:", sum(numbers))
```

Sample Input:

```
How many numbers? 5
Enter number 1: 45
Enter number 2: 12
Enter number 3: 78
Enter number 4: 34
Enter number 5: 56
```

Sample Output

```
List: [45, 12, 78, 34, 56]
Maximum: 78
Minimum: 12
Sum: 225
```

Program 3: Append and Remove Elements

```python
# Program 3: Append and remove elements
fruits = ["apple", "banana", "cherry"]
print("Original:", fruits)

fruits.append("date")
print("After append('date'):", fruits)

fruits.insert(1, "blueberry")
print("After insert(1, 'blueberry'):", fruits)

fruits.extend(["elderberry", "fig"])
print("After extend:", fruits)

fruits.remove("banana")
print("After remove('banana'):", fruits)

popped = fruits.pop()
print("After pop() [removed:", popped + "]:", fruits)
```

```
popped2 = fruits.pop(0)
print("After pop(0) [removed:", popped2 + "]:", fruits)
```

Sample Output

```
Original: ['apple', 'banana', 'cherry']
After append('date'): ['apple', 'banana', 'cherry', 'date']
After insert(1, 'blueberry'): ['apple', 'blueberry', 'banana', '
   cherry', 'date']
After extend: ['apple', 'blueberry', 'banana', 'cherry', 'date',
   'elderberry', 'fig']
After remove('banana'): ['apple', 'blueberry', 'cherry', 'date',
   'elderberry', 'fig']
After pop() [removed: fig]: ['apple', 'blueberry', 'cherry', '
   date', 'elderberry']
After pop(0) [removed: apple]: ['blueberry', 'cherry', 'date', '
   elderberry']
```

Program 4: Create Tuple

```
# Program 4: Create tuple
numbers = (10, 20, 30, 40, 50)
single = (42,)
mixed = (1, "hello", 3.14, True)

print("Numbers tuple:", numbers)
print("Single element tuple:", single)
print("Mixed tuple:", mixed)
print("\nType:", type(numbers))
print("Length:", len(numbers))
print("\nCount of 20:", numbers.count(20))
print("Index of 30:", numbers.index(30))

print("\nTuples are immutable:")
try:
    numbers[0] = 100
except TypeError as e:
    print("Error:", e)
```

Sample Output

```
Numbers tuple: (10, 20, 30, 40, 50)
Single element tuple: (42,)
Mixed tuple: (1, 'hello', 3.14, True)

Type: <class 'tuple'>
Length: 5

Count of 20: 1
Index of 30: 2

Tuples are immutable:
```

```
Error: 'tuple' object does not support item assignment
```

Program 5: Access Elements

```python
# Program 5: Access elements
my_list = [10, 20, 30, 40, 50, 60, 70, 80]
my_tuple = ('a', 'b', 'c', 'd', 'e', 'f')

print("List:", my_list)
print("Tuple:", my_tuple)

print("\n--- Positive Indexing ---")
print("List[0]:", my_list[0])
print("Tuple[2]:", my_tuple[2])

print("\n--- Negative Indexing ---")
print("List[-1]:", my_list[-1])
print("Tuple[-2]:", my_tuple[-2])

print("\n--- Slicing ---")
print("List[2:5]:", my_list[2:5])
print("List[::2]:", my_list[::2])
print("Tuple[:3]:", my_tuple[:3])
print("Tuple[::-1]:", my_tuple[::-1])
```

Sample Output

```
List: [10, 20, 30, 40, 50, 60, 70, 80]
Tuple: ('a', 'b', 'c', 'd', 'e', 'f')

--- Positive Indexing ---
List[0]: 10
Tuple[2]: c

--- Negative Indexing ---
List[-1]: 80
Tuple[-2]: e

--- Slicing ---
List[2:5]: [30, 40, 50]
List[::2]: [10, 30, 50, 70]
Tuple[:3]: ('a', 'b', 'c')
Tuple[::-1]: ('f', 'e', 'd', 'c', 'b', 'a')
```

5.3.2 Medium Programs

Program 6: Remove Duplicates

```python
# Program 6: Remove duplicates
original = [4, 2, 7, 2, 4, 1, 7, 3, 1, 5]
print("Original list:", original)
```

```
unique = list(dict.fromkeys(original))
print("After removing duplicates:", unique)

unique2 = []
for item in original:
    if item not in unique2:
        unique2.append(item)
print("Manual method:", unique2)
```

Sample Output

```
Original list: [4, 2, 7, 2, 4, 1, 7, 3, 1, 5]
After removing duplicates: [4, 2, 7, 1, 3, 5]
Manual method: [4, 2, 7, 1, 3, 5]
```

Program 7: Sort a List

```
# Program 7: Sort a list
numbers = [64, 34, 25, 12, 22, 11, 90]
print("Original:", numbers)

asc = sorted(numbers)
desc = sorted(numbers, reverse=True)
print("Ascending (sorted):", asc)
print("Descending (sorted):", desc)

numbers.sort()
print("After .sort():", numbers)
```

Sample Output

```
Original: [64, 34, 25, 12, 22, 11, 90]
Ascending (sorted): [11, 12, 22, 25, 34, 64, 90]
Descending (sorted): [90, 64, 34, 25, 22, 12, 11]
After .sort(): [11, 12, 22, 25, 34, 64, 90]
```

Program 8: Second Largest Element

```
# Program 8: Second largest element
def second_largest(lst):
    if len(lst) < 2:
        return None
    first = second = float('-inf')
    for num in lst:
        if num > first:
            second = first
            first = num
        elif num > second and num != first:
            second = num
    return second if second != float('-inf') else None
```

```
numbers = [45, 78, 12, 90, 34, 67]
print("List:", numbers)
print("Largest:", max(numbers))
print("Second Largest:", second_largest(numbers))
```

Sample Output

```
List: [45, 78, 12, 90, 34, 67]
Largest: 90
Second Largest: 78
```

Program 9: Tuple Slicing

```
# Program 9: Tuple slicing
t = (10, 20, 30, 40, 50, 60, 70, 80, 90, 100)
print("Tuple:", t)

print("\nSlicing Operations:")
print("t[2:6]:", t[2:6])
print("t[:5]:", t[:5])
print("t[5:]:", t[5:])
print("t[::3]:", t[::3])
print("t[::-1]:", t[::-1])

t1 = (1, 2, 3)
t2 = (4, 5, 6)
print("\nConcatenation:", t1 + t2)
print("Repetition:", t1 * 3)

a, b, c = t1
print("\nUnpacking:", a, b, c)
```

Sample Output

```
Tuple: (10, 20, 30, 40, 50, 60, 70, 80, 90, 100)

Slicing Operations:
t[2:6]: (30, 40, 50, 60)
t[:5]: (10, 20, 30, 40, 50)
t[5:]: (60, 70, 80, 90, 100)
t[::3]: (10, 40, 70, 100)
t[::-1]: (100, 90, 80, 70, 60, 50, 40, 30, 20, 10)

Concatenation: (1, 2, 3, 4, 5, 6)
Repetition: (1, 2, 3, 1, 2, 3, 1, 2, 3)

Unpacking: 1 2 3
```

Program 10: List Comprehension

```python
# Program 10: List comprehension
squares = [x**2 for x in range(1, 11)]
print("Squares:", squares)

evens = [x for x in range(1, 21) if x % 2 == 0]
print("Even numbers:", evens)

celsius = [0, 10, 20, 30, 40]
fahrenheit = [(9/5)*c + 32 for c in celsius]
print("Celsius:", celsius)
print("Fahrenheit:", fahrenheit)

words = ["hello", "world", "python"]
upper_words = [w.upper() for w in words]
print("Uppercase:", upper_words)

matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [num for row in matrix for num in row]
print("Flattened:", flat)
```

Sample Output

```
Squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Even numbers: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
Celsius: [0, 10, 20, 30, 40]
Fahrenheit: [32.0, 50.0, 68.0, 86.0, 104.0]
Uppercase: ['HELLO', 'WORLD', 'PYTHON']
Flattened: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

5.3.3 Advanced Programs

Program 11: Flatten Nested List

```python
# Program 11: Flatten nested list
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result

nested1 = [1, [2, 3], [4, [5, 6]], 7]
nested2 = [[1, 2], [[3, 4], [5, [6, 7]]], 8, [9]]

print("Nested:", nested1)
print("Flattened:", flatten(nested1))
print("\nNested:", nested2)
print("Flattened:", flatten(nested2))
```

Sample Output

```
Nested: [1, [2, 3], [4, [5, 6]], 7]
Flattened: [1, 2, 3, 4, 5, 6, 7]

Nested: [[1, 2], [[3, 4], [5, [6, 7]]], 8, [9]]
Flattened: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Program 12: Tuple-Based Operations

```python
# Program 12: Tuple-based operations
students = [
    ("Alice", 101, 92),
    ("Bob", 102, 78),
    ("Charlie", 103, 95),
    ("Diana", 104, 85),
    ("Eve", 105, 88)
]

print("Student Records:")
print("-" * 40)
print(f"{'Name':<12}{'Roll':<8}{'Marks':<8}")
print("-" * 40)
for name, roll, marks in students:
    print(f"{name:<12}{roll:<8}{marks:<8}")

marks_list = [s[2] for s in students]
print(f"\nHighest Marks: {max(marks_list)}")
print(f"Lowest Marks: {min(marks_list)}")
print(f"Average Marks: {sum(marks_list)/len(marks_list):.2f}")

sorted_students = sorted(students, key=lambda s: s[2], reverse=
   True)
print("\nRank List:")
for rank, (name, roll, marks) in enumerate(sorted_students, 1):
    print(f"  Rank {rank}: {name} ({marks} marks)")
```

Sample Output

```
Student Records:
----------------------------------------
Name         Roll     Marks
----------------------------------------
Alice        101      92
Bob          102      78
Charlie      103      95
Diana        104      85
Eve          105      88

Highest Marks: 95
Lowest Marks: 78
Average Marks: 87.60
```

```
Rank List:
  Rank 1: Charlie (95 marks)
  Rank 2: Alice (92 marks)
  Rank 3: Eve (88 marks)
  Rank 4: Diana (85 marks)
  Rank 5: Bob (78 marks)
```

5.4 Learning Outcomes

Upon completing this lab, students will be able to:

1. Create and manipulate lists using built-in methods.
2. Understand the difference between mutable (list) and immutable (tuple) types.
3. Use list comprehensions for concise code.
4. Apply indexing, slicing, and tuple unpacking.
5. Implement sorting, searching, and filtering on collections.

6 Lab 7: Sets and Dictionaries

6.1 Objective

To understand and implement set and dictionary data structures in Python, including their operations, use cases, and the unique properties that distinguish them from lists and tuples.

6.2 Theory

Sets: A set is an unordered collection of **unique** elements defined using curly braces `{ }` or `set()`. Sets support mathematical operations like union, intersection, difference, and symmetric difference.

Dictionaries: A dictionary is an unordered collection of **key-value pairs** defined using curly braces with `key: value` syntax. Keys must be unique and immutable. Dictionaries provide fast lookup by key.

6.3 Programs

6.3.1 Basic Programs

Program 1: Create a Set

```python
# Program 1: Create a set
fruits = {"apple", "banana", "cherry", "apple", "banana"}
numbers = set([1, 2, 3, 2, 1, 4, 5])
empty_set = set()

print("Fruits:", fruits)
print("Numbers:", numbers)
print("Empty set:", empty_set)
print("Type:", type(fruits))
print("Length:", len(fruits))
print("'apple' in fruits:", "apple" in fruits)
```

Sample Output

```
Fruits: {'cherry', 'banana', 'apple'}
Numbers: {1, 2, 3, 4, 5}
Empty set: set()
Type: <class 'set'>
Length: 3
'apple' in fruits: True
```

Program 2: Add/Remove Elements

```python
# Program 2: Add/Remove elements from set
colors = {"red", "green", "blue"}
print("Original:", colors)
```

```python
colors.add("yellow")
print("After add('yellow'):", colors)

colors.discard("green")
print("After discard('green'):", colors)

colors.remove("blue")
print("After remove('blue'):", colors)

colors.discard("purple")
print("After discard('purple'):", colors)
```

Sample Output

```
Original: {'red', 'green', 'blue'}
After add('yellow'): {'red', 'green', 'blue', 'yellow'}
After discard('green'): {'red', 'blue', 'yellow'}
After remove('blue'): {'red', 'yellow'}
After discard('purple'): {'red', 'yellow'}
```

Program 3: Create Dictionary

```python
# Program 3: Create dictionary
student = {
    "name": "Alice",
    "age": 20,
    "course": "B.Tech CSE",
    "university": "CMR University"
}

print("Student Dictionary:")
for key, value in student.items():
    print(f"  {key}: {value}")
print("\nType:", type(student))
print("Length:", len(student))
```

Sample Output

```
Student Dictionary:
  name: Alice
  age: 20
  course: B.Tech CSE
  university: CMR University

Type: <class 'dict'>
Length: 4
```

Program 4: Access Values

```python
# Program 4: Access dictionary values
student = {"name": "Bob", "age": 21, "grade": "A", "city": "
```

```
    Bengaluru"}

print("Name:", student["name"])
print("Grade:", student.get("grade"))
print("Phone:", student.get("phone", "Not available"))
print("\nKeys:", list(student.keys()))
print("Values:", list(student.values()))
```

Sample Output

```
Name: Bob
Grade: A
Phone: Not available

Keys: ['name', 'age', 'grade', 'city']
Values: ['Bob', 21, 'A', 'Bengaluru']
```

Program 5: Update Dictionary

```
# Program 5: Update dictionary
person = {"name": "Charlie", "age": 22}
print("Original:", person)

person["email"] = "charlie@cmr.edu"
print("After adding email:", person)

person["age"] = 23
print("After updating age:", person)

person.update({"phone": "9876543210", "city": "Bengaluru"})
print("After update():", person)

removed = person.pop("phone")
print("After pop('phone'):", person)
print("Removed value:", removed)
```

Sample Output

```
Original: {'name': 'Charlie', 'age': 22}
After adding email: {'name': 'Charlie', 'age': 22, 'email': '
    charlie@cmr.edu'}
After updating age: {'name': 'Charlie', 'age': 23, 'email': '
    charlie@cmr.edu'}
After update(): {'name': 'Charlie', 'age': 23, 'email': '
    charlie@cmr.edu', 'phone': '9876543210', 'city': 'Bengaluru'}
After pop('phone'): {'name': 'Charlie', 'age': 23, 'email': '
    charlie@cmr.edu', 'city': 'Bengaluru'}
Removed value: 9876543210
```

6.3.2 Medium Programs

Program 6: Union and Intersection

```python
# Program 6: Union and intersection
set_a = {1, 2, 3, 4, 5}
set_b = {4, 5, 6, 7, 8}

print("Set A:", set_a)
print("Set B:", set_b)
print("\nUnion (A | B):", set_a | set_b)
print("Intersection (A & B):", set_a & set_b)
print("Difference (A - B):", set_a - set_b)
print("Difference (B - A):", set_b - set_a)
print("Symmetric Difference (A ^ B):", set_a ^ set_b)
```

Sample Output

```
Set A: {1, 2, 3, 4, 5}
Set B: {4, 5, 6, 7, 8}

Union (A | B): {1, 2, 3, 4, 5, 6, 7, 8}
Intersection (A & B): {4, 5}
Difference (A - B): {1, 2, 3}
Difference (B - A): {8, 6, 7}
Symmetric Difference (A ^ B): {1, 2, 3, 6, 7, 8}
```

Program 7: Frequency Count

```python
# Program 7: Frequency count using dictionary
text = input("Enter a sentence: ")
words = text.lower().split()
freq = {}

for word in words:
    freq[word] = freq.get(word, 0) + 1

print("\nWord Frequency:")
for word, count in sorted(freq.items(), key=lambda x: x[1],
   reverse=True):
    print(f"  '{word}': {count}")
print(f"\nTotal words: {len(words)}")
print(f"Unique words: {len(freq)}")
```

Sample Input:

Enter a sentence: the cat sat on the mat the cat

Sample Output

```
Word Frequency:
  'the': 3
  'cat': 2
```

```
  'sat': 1
  'on': 1
  'mat': 1

Total words: 8
Unique words: 5
```

Program 8: Find Max Value Key

```python
# Program 8: Find key with maximum value
scores = {"Alice": 92, "Bob": 78, "Charlie": 95, "Diana": 88, "
    Eve": 91}

print("Scores:", scores)
max_key = max(scores, key=scores.get)
min_key = min(scores, key=scores.get)

print(f"\nHighest scorer: {max_key} ({scores[max_key]})")
print(f"Lowest scorer: {min_key} ({scores[min_key]})")
print(f"Average: {sum(scores.values())/len(scores):.2f}")
```

Sample Output

```
Scores: {'Alice': 92, 'Bob': 78, 'Charlie': 95, 'Diana': 88, 'Eve
    ': 91}

Highest scorer: Charlie (95)
Lowest scorer: Bob (78)
Average: 88.80
```

Program 9: Remove Duplicates Using Set

```python
# Program 9: Remove duplicates using set
numbers = [5, 3, 8, 3, 5, 1, 8, 2, 1, 7, 2]
print("Original list:", numbers)

unique = list(set(numbers))
print("After removing duplicates:", sorted(unique))
print("Duplicates removed:", len(numbers) - len(unique))
```

Sample Output

```
Original list: [5, 3, 8, 3, 5, 1, 8, 2, 1, 7, 2]
After removing duplicates: [1, 2, 3, 5, 7, 8]
Duplicates removed: 5
```

Program 10: Iterate Dictionary

```python
# Program 10: Iterate dictionary
capitals = {
```

```
    "India": "New Delhi", "USA": "Washington D.C.",
    "UK": "London", "Japan": "Tokyo", "France": "Paris"
}

print("Country Capitals:")
print(f"{'Country':<15}{'Capital':<20}")
print("-" * 35)
for country, capital in capitals.items():
    print(f"{country:<15}{capital:<20}")

search = input("\nSearch for a country: ")
if search in capitals:
    print(f"Capital of {search}: {capitals[search]}")
else:
    print(f"'{search}' not found in the dictionary.")
```

Sample Input:

```
Search for a country: Japan
```

Sample Output

```
Country Capitals:
Country         Capital
-----------------------------------
India           New Delhi
USA             Washington D.C.
UK              London
Japan           Tokyo
France          Paris

Search for a country: Japan
Capital of Japan: Tokyo
```

6.3.3 Advanced Programs

Program 11: Student Marks System

```
# Program 11: Student marks system
students = {}
n = int(input("Enter number of students: "))

for i in range(n):
    name = input(f"\nStudent {i+1} name: ")
    marks = {}
    for subject in ["Math", "Science", "English"]:
        marks[subject] = float(input(f"  {subject} marks: "))
    students[name] = marks

print("\n" + "=" * 55)
print(f"{'Name':<12}{'Math':<10}{'Science':<10}{'English':<10}{'
   Avg':<10}")
print("=" * 55)
```

```python
for name, marks in students.items():
    avg = sum(marks.values()) / len(marks)
    print(f"{name:<12}{marks['Math']:<10}{marks['Science']:<10}{
        marks['English']:<10}{avg:<10.2f}")

topper = max(students, key=lambda s: sum(students[s].values()))
print(f"\nClass Topper: {topper}")
```

Sample Input:

```
Enter number of students: 2
Student 1 name: Alice
  Math marks: 90
  Science marks: 85
  English marks: 92
Student 2 name: Bob
  Math marks: 78
  Science marks: 82
  English marks: 75
```

Sample Output

```
==========================================================
Name         Math       Science    English    Avg
==========================================================
Alice        90.0       85.0       92.0       89.00
Bob          78.0       82.0       75.0       78.33

Class Topper: Alice
```

Program 12: Inventory System

```python
# Program 12: Inventory system
inventory = {
    "laptop": {"price": 50000, "qty": 10},
    "mouse": {"price": 500, "qty": 50},
    "keyboard": {"price": 1500, "qty": 30},
    "monitor": {"price": 15000, "qty": 15}
}

print("===== Inventory Management System =====\n")
print(f"{'Item':<15}{'Price':<10}{'Qty':<8}{'Value':<12}")
print("-" * 45)

total_value = 0
for item, details in inventory.items():
    value = details["price"] * details["qty"]
    total_value += value
    print(f"{item:<15}Rs.{details['price']:<7}{details['qty']:<8}
        Rs.{value}")
```

```python
print("-" * 45)
print(f"{'Total Inventory Value:':<33}Rs.{total_value}")

most_valuable = max(inventory, key=lambda x: inventory[x]["price"
    ] * inventory[x]["qty"])
print(f"\nMost valuable stock: {most_valuable}")
```

Sample Output

```
===== Inventory Management System =====

Item            Price      Qty      Value
---------------------------------------------
laptop          Rs.50000   10       Rs.500000
mouse           Rs.500     50       Rs.25000
keyboard        Rs.1500    30       Rs.45000
monitor         Rs.15000   15       Rs.225000
---------------------------------------------
Total Inventory Value:              Rs.795000

Most valuable stock: laptop
```

6.4 Learning Outcomes

Upon completing this lab, students will be able to:

1. Create and manipulate sets and dictionaries.
2. Perform set operations (union, intersection, difference).
3. Use dictionaries for key-value data storage and retrieval.
4. Implement frequency counting and data aggregation.
5. Design data-driven applications using dictionaries.

7 Lab 8 & 9: Exception Handling

7.1 Objective

To understand and implement exception handling mechanisms in Python using `try`, `except`, `else`, `finally`, and custom exceptions to build robust programs.

7.2 Theory

Exceptions are runtime errors that disrupt the normal flow of a program. Exception handling allows programmers to anticipate and manage these errors gracefully.

try-except Block: The `try` block contains code that might raise an exception. The `except` block contains code that executes when an exception occurs.

else Block: Executes only when no exception occurs in the `try` block.

finally Block: Always executes, regardless of whether an exception occurred. Used for cleanup operations.

Custom Exceptions: Created by inheriting from the `Exception` class and raised using the `raise` keyword.

7.3 Programs

7.3.1 Basic Programs

Program 1: Handle Divide by Zero

```python
# Program 1: Handle divide by zero
num1 = int(input("Enter numerator: "))
num2 = int(input("Enter denominator: "))

try:
    result = num1 / num2
    print(f"{num1} / {num2} = {result}")
except ZeroDivisionError:
    print("Error: Cannot divide by zero!")
```

Sample Input:

```
Enter numerator: 10
Enter denominator: 0
```

Sample Output

```
Enter numerator: 10
Enter denominator: 0
Error: Cannot divide by zero!
```

Program 2: Handle Invalid Input

```python
# Program 2: Handle invalid input
```

```python
try:
    age = int(input("Enter your age: "))
    print(f"Your age is: {age}")
except ValueError:
    print("Error: Please enter a valid integer!")
```

Sample Input:

```
Enter your age: twenty
```

Sample Output

```
Enter your age: twenty
Error: Please enter a valid integer!
```

Program 3: Handle Index Error

```python
# Program 3: Handle index error
numbers = [10, 20, 30, 40, 50]
print("List:", numbers)

try:
    index = int(input("Enter index to access: "))
    print(f"Element at index {index}: {numbers[index]}")
except IndexError:
    print(f"Error: Index out of range! Valid range: 0 to {len(
        numbers)-1}")
except ValueError:
    print("Error: Please enter a valid integer!")
```

Sample Input:

```
Enter index to access: 10
```

Sample Output

```
List: [10, 20, 30, 40, 50]
Enter index to access: 10
Error: Index out of range! Valid range: 0 to 4
```

Program 4: Handle Key Error

```python
# Program 4: Handle key error
student = {"name": "Alice", "age": 20, "course": "CSE"}
print("Student:", student)

try:
    key = input("Enter key to access: ")
    print(f"{key}: {student[key]}")
except KeyError:
    print(f"Error: Key '{key}' not found!")
    print(f"Available keys: {list(student.keys())}")
```

Sample Input:

```
Enter key to access: grade
```

Sample Output

```
Student: {'name': 'Alice', 'age': 20, 'course': 'CSE'}
Enter key to access: grade
Error: Key 'grade' not found!
Available keys: ['name', 'age', 'course']
```

Program 5: Basic try-except

```python
# Program 5: Basic try-except with general exception
try:
    x = int(input("Enter a number: "))
    result = 100 / x
    print(f"100 / {x} = {result}")
except Exception as e:
    print(f"An error occurred: {e}")
    print(f"Error type: {type(e).__name__}")
```

Sample Input:

```
Enter a number: 0
```

Sample Output

```
Enter a number: 0
An error occurred: division by zero
Error type: ZeroDivisionError
```

7.3.2 Medium Programs

Program 6: Multiple Exceptions

```python
# Program 6: Multiple exceptions
def calculate(a, b, operation):
    try:
        a = float(a)
        b = float(b)
        if operation == "+":
            return a + b
        elif operation == "-":
            return a - b
        elif operation == "*":
            return a * b
        elif operation == "/":
            return a / b
        else:
            raise ValueError(f"Unknown operation: {operation}")
    except ValueError as ve:
        print(f"ValueError: {ve}")
    except ZeroDivisionError:
```

```
        print("ZeroDivisionError: Cannot divide by zero!")
    except TypeError as te:
        print(f"TypeError: {te}")
    return None

print("10 + 5 =", calculate("10", "5", "+"))
print("10 / 0 =", calculate("10", "0", "/"))
print("abc + 5 =", calculate("abc", "5", "+"))
print("10 ^ 5 =", calculate("10", "5", "^"))
```

Sample Output

```
10 + 5 = 15.0
ZeroDivisionError: Cannot divide by zero!
10 / 0 = None
ValueError: could not convert string to float: 'abc'
abc + 5 = None
ValueError: Unknown operation: ^
10 ^ 5 = None
```

Program 7: File Handling Exception

```
# Program 7: File handling exception
filename = input("Enter filename to read: ")

try:
    file = open(filename, "r")
    content = file.read()
    print("File contents:")
    print(content)
except FileNotFoundError:
    print(f"Error: File '{filename}' not found!")
except PermissionError:
    print(f"Error: Permission denied to read '{filename}'!")
except IOError as e:
    print(f"IO Error: {e}")
finally:
    print("File operation attempted.")
```

Sample Input:

```
Enter filename to read: nonexistent.txt
```

Sample Output

```
Enter filename to read: nonexistent.txt
Error: File 'nonexistent.txt' not found!
File operation attempted.
```

Program 8: Custom Error Messages

```
# Program 8: Custom error messages
```

```python
def validate_age(age):
    try:
        age = int(age)
        if age < 0:
            raise ValueError("Age cannot be negative!")
        elif age > 150:
            raise ValueError("Age seems unrealistic (>150)!")
        return age
    except ValueError as e:
        print(f"Validation Error: {e}")
        return None

ages = ["25", "-5", "200", "abc", "18"]
for a in ages:
    result = validate_age(a)
    print(f"  Input: '{a}' -> Result: {result}")
```

Sample Output

```
  Input: '25' -> Result: 25
Validation Error: Age cannot be negative!
  Input: '-5' -> Result: None
Validation Error: Age seems unrealistic (>150)!
  Input: '200' -> Result: None
Validation Error: invalid literal for int() with base 10: 'abc'
  Input: 'abc' -> Result: None
  Input: '18' -> Result: 18
```

Program 9: else Block Usage

```python
# Program 9: else block usage
try:
    num = int(input("Enter a positive number: "))
    if num < 0:
        raise ValueError("Number must be positive!")
    result = num ** 0.5
except ValueError as e:
    print(f"Error: {e}")
else:
    print(f"Square root of {num} = {result:.4f}")
    print("No errors occurred!")
finally:
    print("Program execution complete.")
```

Sample Input:

```
Enter a positive number: 25
```

Sample Output

```
Enter a positive number: 25
Square root of 25 = 5.0000
No errors occurred!
```

```
Program execution complete.
```

Program 10: finally Block Usage

```python
# Program 10: finally block usage
def divide_numbers():
    try:
        a = float(input("Enter numerator: "))
        b = float(input("Enter denominator: "))
        result = a / b
    except ZeroDivisionError:
        print("Error: Division by zero!")
        result = None
    except ValueError:
        print("Error: Invalid number!")
        result = None
    else:
        print(f"Result: {a} / {b} = {result}")
    finally:
        print("--- finally block always executes ---")
        if result is not None:
            print(f"Final result stored: {result}")
        else:
            print("No valid result computed.")
    return result

divide_numbers()
```

Sample Input:

```
Enter numerator: 100
Enter denominator: 3
```

Sample Output

```
Enter numerator: 100
Enter denominator: 3
Result: 100.0 / 3.0 = 33.333333333333336
--- finally block always executes ---
Final result stored: 33.333333333333336
```

7.3.3 Advanced Programs

Program 11: Custom Exception

```python
# Program 11: Custom exception
class InsufficientBalanceError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        self.deficit = amount - balance
        super().__init__(
```

```
            f"Cannot withdraw Rs.{amount}. "
            f"Balance: Rs.{balance}. "
            f"Deficit: Rs.{self.deficit}"
        )

class NegativeAmountError(Exception):
    def __init__(self, amount):
        super().__init__(f"Amount cannot be negative: Rs.{amount}
           ")

def withdraw(balance, amount):
    if amount < 0:
        raise NegativeAmountError(amount)
    if amount > balance:
        raise InsufficientBalanceError(balance, amount)
    return balance - amount

balance = 5000
print(f"Current Balance: Rs.{balance}\n")

test_amounts = [1000, 3000, 2500, -500]
for amount in test_amounts:
    try:
        balance = withdraw(balance, amount)
        print(f"Withdrew Rs.{amount} | Remaining: Rs.{balance}")
    except (InsufficientBalanceError, NegativeAmountError) as e:
        print(f"Transaction Failed: {e}")
```

Sample Output

```
Current Balance: Rs.5000

Withdrew Rs.1000 | Remaining: Rs.4000
Withdrew Rs.3000 | Remaining: Rs.1000
Transaction Failed: Cannot withdraw Rs.2500. Balance: Rs.1000.
   Deficit: Rs.1500
Transaction Failed: Amount cannot be negative: Rs.-500
```

Program 12: Banking System with Exception Handling

```
# Program 12: Banking system with exception handling
class BankAccount:
    def __init__(self, name, balance=0):
        self.name = name
        self.balance = balance
        self.transactions = []

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit amount must be positive!")
        self.balance += amount
```

```python
        self.transactions.append(f"Deposited Rs.{amount}")
        print(f"Rs.{amount} deposited successfully.")

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("Withdrawal amount must be positive!
               ")
        if amount > self.balance:
            raise ValueError(f"Insufficient balance! Available:
               Rs.{self.balance}")
        self.balance -= amount
        self.transactions.append(f"Withdrew Rs.{amount}")
        print(f"Rs.{amount} withdrawn successfully.")

    def show_balance(self):
        print(f"Account: {self.name} | Balance: Rs.{self.balance}
           ")

account = BankAccount("Sudharshan", 10000)
account.show_balance()

operations = [
    ("deposit", 5000), ("withdraw", 3000),
    ("withdraw", 20000), ("deposit", -500), ("withdraw", 2000),
]

print()
for op, amount in operations:
    try:
        if op == "deposit":
            account.deposit(amount)
        elif op == "withdraw":
            account.withdraw(amount)
    except ValueError as e:
        print(f"Transaction Error: {e}")
    finally:
        account.show_balance()
    print()

print("Transaction History:")
for t in account.transactions:
    print(f"  - {t}")
```

Sample Output

```
Account: Sudharshan | Balance: Rs.10000

Rs.5000 deposited successfully.
Account: Sudharshan | Balance: Rs.15000

Rs.3000 withdrawn successfully.
Account: Sudharshan | Balance: Rs.12000
```

```
Transaction Error: Insufficient balance! Available: Rs.12000
Account: Sudharshan | Balance: Rs.12000

Transaction Error: Deposit amount must be positive!
Account: Sudharshan | Balance: Rs.12000

Rs.2000 withdrawn successfully.
Account: Sudharshan | Balance: Rs.10000

Transaction History:
  - Deposited Rs.5000
  - Withdrew Rs.3000
  - Withdrew Rs.2000
```

7.4 Learning Outcomes

Upon completing this lab, students will be able to:

1. Handle common Python exceptions using try-except blocks.
2. Use else and finally blocks appropriately.
3. Handle multiple exception types in a single try block.
4. Create and raise custom exceptions for domain-specific error handling.
5. Build robust programs that handle errors gracefully.

8 Lab 10: Classes and Objects

8.1 Objective

To understand the fundamentals of Object-Oriented Programming (OOP) in Python, including class definition, object creation, constructors, instance variables, and methods.

8.2 Theory

Object-Oriented Programming (OOP) organizes code around objects rather than functions. A **class** is a blueprint for creating objects, defining the attributes (data) and methods (behavior).

Key Concepts:

- **Class:** A template that defines object structure and behavior.
- **Object:** An instance of a class with its own data.
- **Constructor (`__init__`):** Initializes instance variables.
- **self:** Reference to the current instance.
- **Instance Variables:** Variables unique to each object.
- **Methods:** Functions defined inside a class.

8.3 Programs

8.3.1 Basic Programs

Program 1: Student Class

```python
# Program 1: Student class
class Student:
    def __init__(self, name, roll, marks):
        self.name = name
        self.roll = roll
        self.marks = marks

    def display(self):
        print(f"Name: {self.name}")
        print(f"Roll: {self.roll}")
        print(f"Marks: {self.marks}")
        print(f"Grade: {self.get_grade()}")

    def get_grade(self):
        if self.marks >= 90: return "A+"
        elif self.marks >= 80: return "A"
        elif self.marks >= 70: return "B"
        elif self.marks >= 60: return "C"
        else: return "F"

s1 = Student("Alice", 101, 92)
```

```python
s2 = Student("Bob", 102, 75)

print("--- Student 1 ---")
s1.display()
print("\n--- Student 2 ---")
s2.display()
```

Sample Output

```
--- Student 1 ---
Name: Alice
Roll: 101
Marks: 92
Grade: A+

--- Student 2 ---
Name: Bob
Roll: 102
Marks: 75
Grade: B
```

Program 2: Car Class

```python
# Program 2: Car class
class Car:
    def __init__(self, brand, model, year, speed=0):
        self.brand = brand
        self.model = model
        self.year = year
        self.speed = speed

    def accelerate(self, increment):
        self.speed += increment
        print(f"{self.brand} {self.model} accelerated to {self.
           speed} km/h")

    def brake(self, decrement):
        self.speed = max(0, self.speed - decrement)
        print(f"{self.brand} {self.model} slowed to {self.speed}
           km/h")

    def display(self):
        print(f"Car: {self.brand} {self.model} ({self.year})")
        print(f"Current Speed: {self.speed} km/h")

car1 = Car("Toyota", "Camry", 2024)
car1.display()
car1.accelerate(60)
car1.accelerate(30)
car1.brake(20)
```

Sample Output

```
Car: Toyota Camry (2024)
Current Speed: 0 km/h
Toyota Camry accelerated to 60 km/h
Toyota Camry accelerated to 90 km/h
Toyota Camry slowed to 70 km/h
```

Program 3: Constructor Example

```python
# Program 3: Constructor example
class Book:
    def __init__(self, title, author, pages, price):
        self.title = title
        self.author = author
        self.pages = pages
        self.price = price
        print(f"Book '{self.title}' created!")

    def __str__(self):
        return f"'{self.title}' by {self.author} ({self.pages}
           pages, Rs.{self.price})"

b1 = Book("Python Programming", "Dr. Babu", 450, 599)
b2 = Book("Data Structures", "Cormen", 1312, 899)

print("\nBook details:")
print(b1)
print(b2)
```

Sample Output

```
Book 'Python Programming' created!
Book 'Data Structures' created!

Book details:
'Python Programming' by Dr. Babu (450 pages, Rs.599)
'Data Structures' by Cormen (1312 pages, Rs.899)
```

Program 4: Instance Variables

```python
# Program 4: Instance variables
class Circle:
    pi = 3.14159  # Class variable

    def __init__(self, radius):
        self.radius = radius  # Instance variable

    def area(self):
        return Circle.pi * self.radius ** 2
```

```
    def circumference(self):
        return 2 * Circle.pi * self.radius

c1 = Circle(5)
c2 = Circle(10)

print(f"Circle 1 (r={c1.radius}): Area={c1.area():.2f},
   Circumference={c1.circumference():.2f}")
print(f"Circle 2 (r={c2.radius}): Area={c2.area():.2f},
   Circumference={c2.circumference():.2f}")
print(f"Pi (class variable): {Circle.pi}")
```

Sample Output

```
Circle 1 (r=5): Area=78.54, Circumference=31.42
Circle 2 (r=10): Area=314.16, Circumference=62.83
Pi (class variable): 3.14159
```

Program 5: Method Usage

```
# Program 5: Method usage
class Calculator:
    def __init__(self):
        self.history = []

    def add(self, a, b):
        result = a + b
        self.history.append(f"{a} + {b} = {result}")
        return result

    def subtract(self, a, b):
        result = a - b
        self.history.append(f"{a} - {b} = {result}")
        return result

    def multiply(self, a, b):
        result = a * b
        self.history.append(f"{a} * {b} = {result}")
        return result

    def show_history(self):
        print("Calculation History:")
        for item in self.history:
            print(f"  {item}")

calc = Calculator()
print("Add:", calc.add(10, 5))
print("Subtract:", calc.subtract(20, 8))
print("Multiply:", calc.multiply(6, 7))
calc.show_history()
```

Sample Output

```
Add: 15
Subtract: 12
Multiply: 42
Calculation History:
  10 + 5 = 15
  20 - 8 = 12
  6 * 7 = 42
```

8.3.2 Medium Programs

Program 6: Bank Account Class

```python
# Program 6: Bank account class
class BankAccount:
    account_count = 0

    def __init__(self, holder, balance=0):
        BankAccount.account_count += 1
        self.account_no = 1000 + BankAccount.account_count
        self.holder = holder
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited Rs.{amount}. Balance: Rs.{self.
                balance}")

    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew Rs.{amount}. Balance: Rs.{self.
                balance}")
        else:
            print("Insufficient balance or invalid amount!")

    def display(self):
        print(f"Acc No: {self.account_no} | Holder: {self.holder}
            | Balance: Rs.{self.balance}")

a1 = BankAccount("Alice", 5000)
a2 = BankAccount("Bob", 3000)

a1.display()
a1.deposit(2000)
a1.withdraw(1500)
print()
a2.display()
a2.withdraw(5000)
```

Sample Output

```
Acc No: 1001 | Holder: Alice | Balance: Rs.5000
Deposited Rs.2000. Balance: Rs.7000
Withdrew Rs.1500. Balance: Rs.5500

Acc No: 1002 | Holder: Bob | Balance: Rs.3000
Insufficient balance or invalid amount!
```

Program 7: Rectangle Class

```python
# Program 7: Rectangle class
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)

    def is_square(self):
        return self.length == self.width

    def display(self):
        print(f"Rectangle ({self.length} x {self.width})")
        print(f"  Area: {self.area()}")
        print(f"  Perimeter: {self.perimeter()}")
        print(f"  Is Square: {self.is_square()}")

r1 = Rectangle(10, 5)
r2 = Rectangle(7, 7)
r1.display()
print()
r2.display()
```

Sample Output

```
Rectangle (10 x 5)
  Area: 50
  Perimeter: 30
  Is Square: False

Rectangle (7 x 7)
  Area: 49
  Perimeter: 28
  Is Square: True
```

Program 8: Employee Class

```python
# Program 8: Employee class
class Employee:
    def __init__(self, name, emp_id, department, salary):
        self.name = name
        self.emp_id = emp_id
        self.department = department
        self.salary = salary

    def annual_salary(self):
        return self.salary * 12

    def apply_raise(self, percentage):
        increment = self.salary * percentage / 100
        self.salary += increment
        print(f"Raise of {percentage}% applied. New salary: Rs.{
           self.salary:.2f}")

    def display(self):
        print(f"ID: {self.emp_id} | Name: {self.name}")
        print(f"Department: {self.department}")
        print(f"Monthly Salary: Rs.{self.salary:.2f}")
        print(f"Annual Salary: Rs.{self.annual_salary():.2f}")

e1 = Employee("Alice", "EMP001", "Engineering", 75000)
e2 = Employee("Bob", "EMP002", "Marketing", 60000)

print("--- Employee 1 ---")
e1.display()
e1.apply_raise(10)

print("\n--- Employee 2 ---")
e2.display()
e2.apply_raise(15)
```

Sample Output

```
--- Employee 1 ---
ID: EMP001 | Name: Alice
Department: Engineering
Monthly Salary: Rs.75000.00
Annual Salary: Rs.900000.00
Raise of 10% applied. New salary: Rs.82500.00

--- Employee 2 ---
ID: EMP002 | Name: Bob
Department: Marketing
Monthly Salary: Rs.60000.00
Annual Salary: Rs.720000.00
Raise of 15% applied. New salary: Rs.69000.00
```

Program 9: Multiple Objects

```python
# Program 9: Multiple objects
class Product:
    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity

    def total_value(self):
        return self.price * self.quantity

    def __str__(self):
        return f"{self.name:<15} Rs.{self.price:<10} Qty:{self.
           quantity:<5} Value: Rs.{self.total_value()}"

products = [
    Product("Laptop", 50000, 10),
    Product("Mouse", 500, 50),
    Product("Keyboard", 1500, 30),
    Product("Monitor", 15000, 15),
    Product("Headphones", 2000, 25)
]

print("===== Product Inventory =====")
for product in products:
    print(product)

total = sum(p.total_value() for p in products)
print(f"\nTotal Inventory Value: Rs.{total}")
```

Sample Output

```
===== Product Inventory =====
Laptop          Rs.50000      Qty:10    Value: Rs.500000
Mouse           Rs.500        Qty:50    Value: Rs.25000
Keyboard        Rs.1500       Qty:30    Value: Rs.45000
Monitor         Rs.15000      Qty:15    Value: Rs.225000
Headphones      Rs.2000       Qty:25    Value: Rs.50000

Total Inventory Value: Rs.845000
```

Program 10: Default Values

```python
# Program 10: Default values
class UserProfile:
    def __init__(self, username, email, age=18, city="Unknown",
       role="User"):
        self.username = username
        self.email = email
        self.age = age
        self.city = city
        self.role = role
```

```
    def display(self):
        print(f"Username: {self.username}")
        print(f"Email: {self.email}")
        print(f"Age: {self.age}")
        print(f"City: {self.city}")
        print(f"Role: {self.role}")

u1 = UserProfile("alice", "alice@cmr.edu")
u2 = UserProfile("bob", "bob@cmr.edu", 21, "Bengaluru")
u3 = UserProfile("charlie", "charlie@cmr.edu", 25, "Mumbai", "
   Admin")

print("--- Profile 1 (defaults used) ---")
u1.display()
print("\n--- Profile 2 (partial) ---")
u2.display()
print("\n--- Profile 3 (all specified) ---")
u3.display()
```

Sample Output

```
--- Profile 1 (defaults used) ---
Username: alice
Email: alice@cmr.edu
Age: 18
City: Unknown
Role: User

--- Profile 2 (partial) ---
Username: bob
Email: bob@cmr.edu
Age: 21
City: Bengaluru
Role: User

--- Profile 3 (all specified) ---
Username: charlie
Email: charlie@cmr.edu
Age: 25
City: Mumbai
Role: Admin
```

8.3.3 Advanced Programs

Program 11: Library System

```
# Program 11: Library system
class LibraryBook:
    def __init__(self, title, author):
        self.title = title
        self.author = author
```

```python
        self.is_available = True

    def __str__(self):
        status = "Available" if self.is_available else "Issued"
        return f"'{self.title}' by {self.author} [{status}]"

class Library:
    def __init__(self, name):
        self.name = name
        self.books = []

    def add_book(self, title, author):
        book = LibraryBook(title, author)
        self.books.append(book)
        print(f"Added: '{title}' by {author}")

    def issue_book(self, title):
        for book in self.books:
            if book.title == title:
                if book.is_available:
                    book.is_available = False
                    print(f"Issued: '{title}'")
                else:
                    print(f"'{title}' is already issued!")
                return
        print(f"'{title}' not found!")

    def return_book(self, title):
        for book in self.books:
            if book.title == title:
                if not book.is_available:
                    book.is_available = True
                    print(f"Returned: '{title}'")
                else:
                    print(f"'{title}' was not issued!")
                return
        print(f"'{title}' not found!")

    def display_books(self):
        print(f"\n===== {self.name} =====")
        for i, book in enumerate(self.books, 1):
            print(f"  {i}. {book}")

lib = Library("CMR University Library")
lib.add_book("Python Programming", "Dr. Babu")
lib.add_book("Data Structures", "Cormen")
lib.add_book("Machine Learning", "Andrew Ng")
lib.display_books()

lib.issue_book("Python Programming")
lib.issue_book("Python Programming")
```

```
lib.return_book("Python Programming")
lib.display_books()
```

Sample Output

```
Added: 'Python Programming' by Dr. Babu
Added: 'Data Structures' by Cormen
Added: 'Machine Learning' by Andrew Ng

===== CMR University Library =====
  1. 'Python Programming' by Dr. Babu [Available]
  2. 'Data Structures' by Cormen [Available]
  3. 'Machine Learning' by Andrew Ng [Available]
Issued: 'Python Programming'
'Python Programming' is already issued!
Returned: 'Python Programming'

===== CMR University Library =====
  1. 'Python Programming' by Dr. Babu [Available]
  2. 'Data Structures' by Cormen [Available]
  3. 'Machine Learning' by Andrew Ng [Available]
```

Program 12: ATM Simulation

```
# Program 12: ATM simulation
class ATM:
    def __init__(self, holder, balance, pin):
        self.holder = holder
        self.balance = balance
        self.__pin = pin

    def verify_pin(self, entered_pin):
        return self.__pin == entered_pin

    def check_balance(self, pin):
        if self.verify_pin(pin):
            print(f"Account Holder: {self.holder}")
            print(f"Current Balance: Rs.{self.balance:.2f}")
        else:
            print("Incorrect PIN!")

    def deposit(self, pin, amount):
        if not self.verify_pin(pin):
            print("Incorrect PIN!")
            return
        if amount <= 0:
            print("Invalid deposit amount!")
            return
        self.balance += amount
        print(f"Rs.{amount:.2f} deposited. New Balance: Rs.{self.
           balance:.2f}")
```

```python
    def withdraw(self, pin, amount):
        if not self.verify_pin(pin):
            print("Incorrect PIN!")
            return
        if amount > self.balance:
            print("Insufficient balance!")
            return
        self.balance -= amount
        print(f"Rs.{amount:.2f} withdrawn. Remaining: Rs.{self.
           balance:.2f}")

    def change_pin(self, old_pin, new_pin):
        if not self.verify_pin(old_pin):
            print("Incorrect old PIN!")
            return
        self.__pin = new_pin
        print("PIN changed successfully!")

atm = ATM("Sudharshan", 50000, 1234)
print("===== ATM Simulation =====\n")
atm.check_balance(1234)
print()
atm.deposit(1234, 10000)
atm.withdraw(1234, 25000)
print()
atm.withdraw(9999, 1000)
print()
atm.change_pin(1234, 5678)
atm.check_balance(5678)
```

Sample Output

```
===== ATM Simulation =====

Account Holder: Sudharshan
Current Balance: Rs.50000.00

Rs.10000.00 deposited. New Balance: Rs.60000.00
Rs.25000.00 withdrawn. Remaining: Rs.35000.00

Incorrect PIN!

PIN changed successfully!
Account Holder: Sudharshan
Current Balance: Rs.35000.00
```

8.4 Learning Outcomes

Upon completing this lab, students will be able to:

1. Define classes with constructors and instance variables.

2. Create objects and call methods on them.
3. Understand class variables vs instance variables.
4. Use `__str__` and `__init__` special methods.
5. Design object-oriented solutions for real-world problems.

9 Lab 11: Static Polymorphism

9.1 Objective

To understand and implement static polymorphism in Python using default arguments, `*args`, `**kwargs`, and conditional logic to achieve method behavior that varies based on input.

9.2 Theory

Polymorphism means "many forms." Static polymorphism is achieved at compile-time. Python does not support traditional method overloading, but achieves similar behavior through:

- **Default Arguments:** Parameters with default values.
- ***args:** Accepts any number of positional arguments as a tuple.
- ****kwargs:** Accepts any number of keyword arguments as a dictionary.
- **Type Checking:** Using `isinstance()` for type-based behavior.

9.3 Programs

9.3.1 Basic Programs

Program 1: Default Arguments

```python
# Program 1: Default arguments
def greet(name, greeting="Hello", punctuation="!"):
    print(f"{greeting}, {name}{punctuation}")

greet("Alice")
greet("Bob", "Good morning")
greet("Charlie", "Hi", "!!!")
greet("Diana", punctuation=".")
```

Sample Output

```
Hello, Alice!
Good morning, Bob!
Hi, Charlie!!!
Hello, Diana.
```

Program 2: Function Overloading Simulation

```python
# Program 2: Function overloading simulation
def calculate_area(length=None, breadth=None, radius=None):
    if radius is not None:
        area = 3.14159 * radius ** 2
        print(f"Circle with radius {radius}: Area = {area:.2f}")
    elif length is not None and breadth is not None:
        area = length * breadth
```

```
        print(f"Rectangle ({length} x {breadth}): Area = {area:.2
            f}")
    elif length is not None:
        area = length ** 2
        print(f"Square with side {length}: Area = {area:.2f}")
    else:
        print("Error: Please provide valid dimensions!")

calculate_area(length=5)
calculate_area(length=10, breadth=5)
calculate_area(radius=7)
calculate_area()
```

Sample Output

```
Square with side 5: Area = 25.00
Rectangle (10 x 5): Area = 50.00
Circle with radius 7: Area = 153.94
Error: Please provide valid dimensions!
```

Program 3: Add Function

```
# Program 3: Add function
def add(a, b, c=0, d=0):
    result = a + b + c + d
    nums = [a, b]
    if c != 0: nums.append(c)
    if d != 0: nums.append(d)
    print(f"Sum of {nums} = {result}")
    return result

add(10, 20)
add(10, 20, 30)
add(10, 20, 30, 40)
```

Sample Output

```
Sum of [10, 20] = 30
Sum of [10, 20, 30] = 60
Sum of [10, 20, 30, 40] = 100
```

Program 4: Multiply Function

```
# Program 4: Multiply function
def multiply(a, b=1, c=1, d=1):
    return a * b * c * d

print("multiply(5):", multiply(5))
print("multiply(5, 3):", multiply(5, 3))
print("multiply(5, 3, 2):", multiply(5, 3, 2))
print("multiply(5, 3, 2, 4):", multiply(5, 3, 2, 4))
```

Sample Output

```
multiply(5): 5
multiply(5, 3): 15
multiply(5, 3, 2): 30
multiply(5, 3, 2, 4): 120
```

Program 5: Conditional Logic Method

```python
# Program 5: Conditional logic method
class Formatter:
    def format_data(self, data):
        if isinstance(data, str):
            print(f"String: '{data}' (length: {len(data)})")
        elif isinstance(data, int):
            print(f"Integer: {data} (even: {data % 2 == 0})")
        elif isinstance(data, list):
            print(f"List: {data} (elements: {len(data)})")
        elif isinstance(data, dict):
            print(f"Dictionary: {len(data)} key-value pairs")
            for k, v in data.items():
                print(f"  {k}: {v}")
        else:
            print(f"Unknown type: {type(data).__name__}")

f = Formatter()
f.format_data("Hello Python")
f.format_data(42)
f.format_data([1, 2, 3, 4, 5])
f.format_data({"name": "Alice", "age": 20})
f.format_data(3.14)
```

Sample Output

```
String: 'Hello Python' (length: 12)
Integer: 42 (even: True)
List: [1, 2, 3, 4, 5] (elements: 5)
Dictionary: 2 key-value pairs
  name: Alice
  age: 20
Unknown type: float
```

9.3.2 Medium Programs

Program 6: Area Calculation

```python
# Program 6: Area calculation
class ShapeCalculator:
    def area(self, shape, *dimensions):
        if shape == "circle" and len(dimensions) == 1:
            r = dimensions[0]
```

```
            print(f"Circle (r={r}): Area = {3.14159 * r ** 2:.2f}
                ")
        elif shape == "rectangle" and len(dimensions) == 2:
            l, w = dimensions
            print(f"Rectangle ({l} x {w}): Area = {l * w:.2f}")
        elif shape == "triangle" and len(dimensions) == 2:
            b, h = dimensions
            print(f"Triangle (b={b}, h={h}): Area = {0.5 * b * h
                :.2f}")
        elif shape == "square" and len(dimensions) == 1:
            s = dimensions[0]
            print(f"Square (s={s}): Area = {s ** 2:.2f}")
        else:
            print(f"Cannot calculate area for '{shape}'.")

calc = ShapeCalculator()
calc.area("circle", 7)
calc.area("rectangle", 10, 5)
calc.area("triangle", 8, 6)
calc.area("square", 4)
```

Sample Output

```
Circle (r=7): Area = 153.94
Rectangle (10 x 5): Area = 50.00
Triangle (b=8, h=6): Area = 24.00
Square (s=4): Area = 16.00
```

Program 7: Calculator Class

```
# Program 7: Calculator class
class SmartCalculator:
    def compute(self, a, b, operation="add"):
        ops = {
            "add": a + b, "subtract": a - b,
            "multiply": a * b, "power": a ** b,
            "modulus": a % b
        }
        if operation == "divide":
            if b == 0:
                print("Error: Division by zero!")
                return None
            result = a / b
        elif operation in ops:
            result = ops[operation]
        else:
            print(f"Unknown operation: {operation}")
            return None
        print(f"{a} {operation} {b} = {result}")
        return result
```

```
calc = SmartCalculator()
calc.compute(10, 5)
calc.compute(10, 5, "subtract")
calc.compute(10, 5, "multiply")
calc.compute(10, 5, "divide")
calc.compute(2, 10, "power")
```

Sample Output

```
10 add 5 = 15
10 subtract 5 = 5
10 multiply 5 = 50
10 divide 5 = 2.0
2 power 10 = 1024
```

Program 8: *args Usage

```
# Program 8: *args usage
def flexible_sum(*args):
    print(f"Arguments: {args} -> Sum: {sum(args)}")
    return sum(args)

def average(*args):
    if len(args) == 0:
        return 0
    return sum(args) / len(args)

def concatenate(*args, separator=" "):
    return separator.join(str(a) for a in args)

flexible_sum(10, 20)
flexible_sum(10, 20, 30, 40)
flexible_sum(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

print("Average of 85, 90, 78:", average(85, 90, 78))
print("Concatenate:", concatenate("Python", "is", "awesome"))
print("With dash:", concatenate("A", "B", "C", separator="-"))
```

Sample Output

```
Arguments: (10, 20) -> Sum: 30
Arguments: (10, 20, 30, 40) -> Sum: 100
Arguments: (1, 2, 3, 4, 5, 6, 7, 8, 9, 10) -> Sum: 55
Average of 85, 90, 78: 84.33333333333333
Concatenate: Python is awesome
With dash: A-B-C
```

Program 9: **kwargs Usage

```
# Program 9: **kwargs usage
def build_profile(name, **kwargs):
```

```
    profile = {"name": name}
    profile.update(kwargs)
    print("Profile:")
    for key, value in profile.items():
        print(f"  {key}: {value}")
    print()

def configure(**kwargs):
    defaults = {"theme": "light", "font_size": 12, "language": "
       English"}
    defaults.update(kwargs)
    print("Configuration:")
    for key, value in defaults.items():
        print(f"  {key}: {value}")
    print()

build_profile("Alice", age=20, city="Bengaluru", course="CSE")
build_profile("Bob", age=21, hobby="coding")
configure()
configure(theme="dark", font_size=16)
```

Sample Output

```
Profile:
  name: Alice
  age: 20
  city: Bengaluru
  course: CSE

Profile:
  name: Bob
  age: 21
  hobby: coding

Configuration:
  theme: light
  font_size: 12
  language: English

Configuration:
  theme: dark
  font_size: 16
  language: English
```

Program 10: Method Variations

```
# Program 10: Method variations
class DataProcessor:
    def process(self, data, operation="display"):
        if operation == "display":
            print(f"Data: {data}")
```

```
        elif operation == "reverse":
            if isinstance(data, (str, list)):
                print(f"Reversed: {data[::-1]}")
            else:
                print(f"Cannot reverse type: {type(data).__name__
                    }")
        elif operation == "count":
            print(f"Length/Count: {len(data)}")
        elif operation == "uppercase" and isinstance(data, str):
            print(f"Uppercase: {data.upper()}")
        elif operation == "sort" and isinstance(data, list):
            print(f"Sorted: {sorted(data)}")

dp = DataProcessor()
dp.process("Hello Python")
dp.process("Hello Python", "reverse")
dp.process("Hello Python", "uppercase")
dp.process([5, 2, 8, 1, 9], "sort")
dp.process([5, 2, 8, 1, 9], "reverse")
```

Sample Output

```
Data: Hello Python
Reversed: nohtyP olleH
Uppercase: HELLO PYTHON
Sorted: [1, 2, 5, 8, 9]
Reversed: [9, 1, 8, 2, 5]
```

9.3.3 Advanced Programs

Program 11: Multi-Operation Class

```
# Program 11: Multi-operation class
class MathOps:
    def operate(self, *args, operation="sum", **kwargs):
        if operation == "sum":
            print(f"Sum of {args}: {sum(args)}")
        elif operation == "product":
            result = 1
            for num in args:
                result *= num
            print(f"Product of {args}: {result}")
        elif operation == "average":
            if len(args) == 0:
                print("No numbers provided!")
                return
            print(f"Average of {args}: {sum(args)/len(args):.2f}"
                )
        elif operation == "power" and len(args) == 2:
            print(f"{args[0]} ^ {args[1]} = {args[0] ** args[1]}"
                )
        elif operation == "stats" and len(args) > 0:
```

```
            precision = kwargs.get("precision", 2)
            print(f"Stats for {args}: Min={min(args)}, Max={max(
                args)}, Avg={sum(args)/len(args):.{precision}f}")

m = MathOps()
m.operate(1, 2, 3, 4, 5)
m.operate(1, 2, 3, 4, 5, operation="product")
m.operate(85, 90, 78, 92, operation="average")
m.operate(2, 10, operation="power")
m.operate(10, 20, 30, 40, 50, operation="stats", precision=4)
```

Sample Output

```
Sum of (1, 2, 3, 4, 5): 15
Product of (1, 2, 3, 4, 5): 120
Average of (85, 90, 78, 92): 86.25
2 ^ 10 = 1024
Stats for (10, 20, 30, 40, 50): Min=10, Max=50, Avg=30.0000
```

Program 12: Dynamic Argument Handling

```
# Program 12: Dynamic argument handling
class ReportGenerator:
    def generate(self, *args, **kwargs):
        title = kwargs.get("title", "Report")
        border = kwargs.get("border", "=")
        width = kwargs.get("width", 40)

        print(border * width)
        print(f"{title:^{width}}")
        print(border * width)

        if all(isinstance(a, (int, float)) for a in args) and len
            (args) > 0:
            print(f"  Numbers: {args}")
            print(f"  Sum: {sum(args)}, Avg: {sum(args)/len(args)
                :.2f}")
        elif all(isinstance(a, str) for a in args) and len(args)
            > 0:
            print(f"  Items ({len(args)}):")
            for i, item in enumerate(args, 1):
                print(f"    {i}. {item}")

        extra = {k: v for k, v in kwargs.items() if k not in ["
            title", "border", "width"]}
        if extra:
            print(f"  Info:")
            for key, value in extra.items():
                print(f"    {key}: {value}")
        print(border * width)
        print()
```

```
rg = ReportGenerator()
rg.generate(85, 90, 78, 92, 88, title="Student Marks", border="-"
    )
rg.generate("Python", "Java", "C++", title="Languages")
rg.generate(title="Project", author="Dr. Babu", status="Done")
```

Sample Output

```
----------------------------------------
            Student Marks
----------------------------------------
  Numbers: (85, 90, 78, 92, 88)
  Sum: 433, Avg: 86.60
----------------------------------------

========================================
               Languages
========================================
  Items (3):
    1. Python
    2. Java
    3. C++
========================================

========================================
                Project
========================================
  Info:
    author: Dr. Babu
    status: Done
========================================
```

9.4 Learning Outcomes

Upon completing this lab, students will be able to:

1. Use default arguments to simulate function overloading.
2. Apply `*args` for variable-length positional arguments.
3. Apply `**kwargs` for variable-length keyword arguments.
4. Use `isinstance()` for type-based polymorphism.
5. Design flexible methods that handle multiple argument patterns.

10 Lab 12: Inheritance

10.1 Objective

To understand and implement inheritance in Python, including single, multilevel, and multiple inheritance, method overriding, and the use of `super()`.

10.2 Theory

Inheritance allows a new class (child) to inherit attributes and methods from an existing class (parent). Types include single, multiple, multilevel, hierarchical, and hybrid inheritance.

super(): Returns a proxy object that allows access to parent class methods.

Method Overriding: Child class defines a method with the same name as the parent's, replacing its behavior.

MRO: Python uses C3 Linearization to determine method resolution order.

10.3 Programs

10.3.1 Basic Programs

Program 1: Single Inheritance

```python
# Program 1: Single inheritance
class Animal:
    def __init__(self, name, species):
        self.name = name
        self.species = species

    def speak(self):
        print(f"{self.name} makes a sound.")

    def info(self):
        print(f"Name: {self.name}, Species: {self.species}")

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name, "Dog")
        self.breed = breed

    def bark(self):
        print(f"{self.name} says: Woof! Woof!")

    def display(self):
        self.info()
        print(f"Breed: {self.breed}")

dog = Dog("Buddy", "Golden Retriever")
dog.display()
```

```
dog.speak()
dog.bark()
```

Sample Output

```
Name: Buddy, Species: Dog
Breed: Golden Retriever
Buddy makes a sound.
Buddy says: Woof! Woof!
```

Program 2: Parent-Child Example

```
# Program 2: Parent-child class
class Vehicle:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

    def describe(self):
        print(f"Vehicle: {self.year} {self.make} {self.model}")

    def start(self):
        print(f"The {self.make} {self.model} engine starts.")

class Car(Vehicle):
    def __init__(self, make, model, year, doors):
        super().__init__(make, model, year)
        self.doors = doors

    def describe(self):
        super().describe()
        print(f"Type: Car with {self.doors} doors")

car = Car("Toyota", "Camry", 2023, 4)
car.describe()
car.start()
```

Sample Output

```
Vehicle: 2023 Toyota Camry
Type: Car with 4 doors
The Toyota Camry engine starts.
```

Program 3: Method Reuse

```
# Program 3: Method reuse via inheritance
class Shape:
    def __init__(self, color="white"):
        self.color = color
```

```python
    def describe(self):
        print(f"I am a {self.color} shape.")

    def area(self):
        return 0

class Circle(Shape):
    def __init__(self, radius, color="white"):
        super().__init__(color)
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2

c = Circle(7, "blue")
c.describe()
print(f"Radius: {c.radius}")
print(f"Area: {c.area():.2f}")
```

Sample Output

```
I am a blue shape.
Radius: 7
Area: 153.94
```

Program 4: Attribute Access

```python
# Program 4: Accessing parent attributes from child
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        print(f"Hello, I am {self.name}, aged {self.age}.")

class Student(Person):
    def __init__(self, name, age, roll_no, branch):
        super().__init__(name, age)
        self.roll_no = roll_no
        self.branch = branch

    def display(self):
        self.greet()
        print(f"Roll No: {self.roll_no}, Branch: {self.branch}")

s = Student("Ravi", 19, "CSE101", "Computer Science")
s.display()
print(f"Name from child: {s.name}")
print(f"Age from child: {s.age}")
```

Sample Output

```
Hello, I am Ravi, aged 19.
Roll No: CSE101, Branch: Computer Science
Name from child: Ravi
Age from child: 19
```

Program 5: Simple Inheritance Program

```python
# Program 5: Simple inheritance with constructor and method
class Bank:
    def __init__(self, bank_name):
        self.bank_name = bank_name

    def info(self):
        print(f"Bank: {self.bank_name}")

class SavingsAccount(Bank):
    def __init__(self, bank_name, account_no, balance):
        super().__init__(bank_name)
        self.account_no = account_no
        self.balance = balance

    def display(self):
        self.info()
        print(f"Account No: {self.account_no}")
        print(f"Balance: Rs. {self.balance:.2f}")

acc = SavingsAccount("CMR Bank", "SB00123", 15000.00)
acc.display()
```

Sample Output

```
Bank: CMR Bank
Account No: SB00123
Balance: Rs. 15000.00
```

10.3.2 Medium Programs

Program 6: Multilevel Inheritance

```python
# Program 6: Multilevel inheritance
class LivingBeing:
    def breathe(self):
        print("Living beings breathe.")

class Animal(LivingBeing):
    def __init__(self, name):
        self.name = name

    def eat(self):
        print(f"{self.name} eats food.")
```

```python
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed

    def bark(self):
        print(f"{self.name} ({self.breed}) barks: Woof!")

dog = Dog("Tommy", "Labrador")
dog.breathe()
dog.eat()
dog.bark()
print(f"Is Dog an Animal? {isinstance(dog, Animal)}")
print(f"Is Dog a LivingBeing? {isinstance(dog, LivingBeing)}")
```

Sample Output

```
Living beings breathe.
Tommy eats food.
Tommy (Labrador) barks: Woof!
Is Dog an Animal? True
Is Dog a LivingBeing? True
```

Program 7: Multiple Inheritance

```python
# Program 7: Multiple inheritance
class Flyable:
    def fly(self):
        print(f"{self.name} can fly.")

class Swimmable:
    def swim(self):
        print(f"{self.name} can swim.")

class Duck(Flyable, Swimmable):
    def __init__(self, name):
        self.name = name

    def quack(self):
        print(f"{self.name} says: Quack!")

d = Duck("Donald")
d.fly()
d.swim()
d.quack()
print(f"MRO: {[cls.__name__ for cls in Duck.__mro__]}")
```

Sample Output

```
Donald can fly.
Donald can swim.
```

```
Donald says: Quack!
MRO: ['Duck', 'Flyable', 'Swimmable', 'object']
```

Program 8: super() Usage

```python
# Program 8: Demonstrating super()
class A:
    def __init__(self, x):
        self.x = x
        print(f"A.__init__: x = {x}")

    def show(self):
        print(f"Class A: x = {self.x}")

class B(A):
    def __init__(self, x, y):
        super().__init__(x)
        self.y = y
        print(f"B.__init__: y = {y}")

    def show(self):
        super().show()
        print(f"Class B: y = {self.y}")

class C(B):
    def __init__(self, x, y, z):
        super().__init__(x, y)
        self.z = z
        print(f"C.__init__: z = {z}")

    def show(self):
        super().show()
        print(f"Class C: z = {self.z}")

obj = C(10, 20, 30)
print()
obj.show()
```

Sample Output

```
A.__init__: x = 10
B.__init__: y = 20
C.__init__: z = 30

Class A: x = 10
Class B: y = 20
Class C: z = 30
```

Program 9: Method Overriding

```python
# Program 9: Method overriding
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def calculate_bonus(self):
        return self.salary * 0.10

    def display(self):
        bonus = self.calculate_bonus()
        print(f"Employee: {self.name}")
        print(f"Salary: Rs. {self.salary:.2f}")
        print(f"Bonus (10%): Rs. {bonus:.2f}")
        print(f"Total: Rs. {self.salary + bonus:.2f}")

class Manager(Employee):
    def calculate_bonus(self):  # overrides parent method
        return self.salary * 0.25

class Director(Employee):
    def calculate_bonus(self):  # overrides parent method
        return self.salary * 0.40

emp = Employee("Kiran", 40000)
mgr = Manager("Priya", 80000)
drt = Director("Suresh", 150000)

for person in [emp, mgr, drt]:
    person.display()
    print()
```

Sample Output

```
Employee: Kiran
Salary: Rs. 40000.00
Bonus (10%): Rs. 4000.00
Total: Rs. 44000.00

Employee: Priya
Salary: Rs. 80000.00
Bonus (25%): Rs. 20000.00
Total: Rs. 100000.00

Employee: Suresh
Salary: Rs. 150000.00
Bonus (40%): Rs. 60000.00
Total: Rs. 210000.00
```

Program 10: Employee Hierarchy

```python
# Program 10: Employee hierarchy using inheritance
class Staff:
    def __init__(self, emp_id, name, department):
        self.emp_id = emp_id
        self.name = name
        self.department = department

    def show_basic(self):
        print(f"ID: {self.emp_id} | Name: {self.name} | Dept: {
            self.department}")

class TeachingStaff(Staff):
    def __init__(self, emp_id, name, department, subject,
        qualification):
        super().__init__(emp_id, name, department)
        self.subject = subject
        self.qualification = qualification

    def display(self):
        self.show_basic()
        print(f"  Subject: {self.subject} | Qual: {self.
            qualification}")

class NonTeachingStaff(Staff):
    def __init__(self, emp_id, name, department, role):
        super().__init__(emp_id, name, department)
        self.role = role

    def display(self):
        self.show_basic()
        print(f"  Role: {self.role}")

t1 = TeachingStaff("T001", "Dr. Babu", "CSE", "Python", "Ph.D")
t2 = TeachingStaff("T002", "Prof. Meena", "CSE", "DBMS", "M.Tech"
    )
n1 = NonTeachingStaff("N001", "Ramesh", "Admin", "Clerk")

print("=== Teaching Staff ===")
t1.display()
t2.display()
print("=== Non-Teaching Staff ===")
n1.display()
```

Sample Output

```
=== Teaching Staff ===
ID: T001 | Name: Dr. Babu | Dept: CSE
  Subject: Python | Qual: Ph.D
ID: T002 | Name: Prof. Meena | Dept: CSE
  Subject: DBMS | Qual: M.Tech
=== Non-Teaching Staff ===
ID: N001 | Name: Ramesh | Dept: Admin
```

```
Role: Clerk
```

10.3.3 Advanced Programs

Program 11: Complex Inheritance System

```python
# Program 11: Complex inheritance - University system
class University:
    university_name = "CMR University"

    def __init__(self, usn, name):
        self.usn = usn
        self.name = name

    def display_university(self):
        print(f"University: {University.university_name}")

class Department(University):
    def __init__(self, usn, name, dept, semester):
        super().__init__(usn, name)
        self.dept = dept
        self.semester = semester

    def display_dept(self):
        self.display_university()
        print(f"Department: {self.dept} | Semester: {self.
            semester}")

class AcademicRecord(Department):
    def __init__(self, usn, name, dept, semester, subjects, marks
        ):
        super().__init__(usn, name, dept, semester)
        self.subjects = subjects
        self.marks = marks

    def calculate_sgpa(self):
        avg = sum(self.marks) / len(self.marks)
        return round(avg / 10, 2)

    def display(self):
        self.display_dept()
        print(f"USN: {self.usn} | Name: {self.name}")
        print("Marks:")
        for sub, mark in zip(self.subjects, self.marks):
            print(f"  {sub}: {mark}/100")
        print(f"SGPA: {self.calculate_sgpa()}")

subjects = ["Python", "DBMS", "OS", "CN", "Maths"]
marks = [88, 92, 78, 85, 91]
s = AcademicRecord("1CR23CS001", "Anjali", "CSE", "III", subjects
    , marks)
```

```
s.display()
```

Sample Output

```
University: CMR University
Department: CSE | Semester: III
USN: 1CR23CS001 | Name: Anjali
Marks:
  Python: 88/100
  DBMS: 92/100
  OS: 78/100
  CN: 85/100
  Maths: 91/100
SGPA: 8.68
```

Program 12: Real-World Hierarchy Model

```
# Program 12: Real-world hierarchy - E-commerce system
class Product:
    def __init__(self, product_id, name, price):
        self.product_id = product_id
        self.name = name
        self.price = price

    def display(self):
        print(f"[{self.product_id}] {self.name} - Rs. {self.price
            :.2f}")

class Electronics(Product):
    def __init__(self, product_id, name, price, brand,
        warranty_years):
        super().__init__(product_id, name, price)
        self.brand = brand
        self.warranty_years = warranty_years

    def display(self):
        super().display()
        print(f"  Brand: {self.brand} | Warranty: {self.
            warranty_years} year(s)")

class Laptop(Electronics):
    def __init__(self, product_id, name, price, brand, warranty,
        ram, storage):
        super().__init__(product_id, name, price, brand, warranty
            )
        self.ram = ram
        self.storage = storage

    def display(self):
        super().display()
        print(f"  RAM: {self.ram}GB | Storage: {self.storage}GB")
```

```python
        print(f"  Price after 10% discount: Rs. {self.price *
            0.90:.2f}")

class Clothing(Product):
    def __init__(self, product_id, name, price, size, fabric):
        super().__init__(product_id, name, price)
        self.size = size
        self.fabric = fabric

    def display(self):
        super().display()
        print(f"  Size: {self.size} | Fabric: {self.fabric}")

cart = [
    Laptop("E001", "Dell Inspiron 15", 65000, "Dell", 2, 16, 512)
        ,
    Clothing("C001", "Cotton Shirt", 850, "M", "Cotton"),
    Electronics("E002", "Sony Headphones", 3500, "Sony", 1),
]

print("=== Shopping Cart ===")
for item in cart:
    item.display()
    print()
```

Sample Output

```
=== Shopping Cart ===
[E001] Dell Inspiron 15 - Rs. 65000.00
  Brand: Dell | Warranty: 2 year(s)
  RAM: 16GB | Storage: 512GB
  Price after 10% discount: Rs. 58500.00

[C001] Cotton Shirt - Rs. 850.00
  Size: M | Fabric: Cotton

[E002] Sony Headphones - Rs. 3500.00
  Brand: Sony | Warranty: 1 year(s)
```

10.4 Learning Outcomes

Upon completing this lab, students will be able to:

1. Implement single, multilevel, and multiple inheritance in Python.
2. Use `super()` to access and extend parent class behavior.
3. Override methods in child classes to customize behavior.
4. Understand Method Resolution Order (MRO) in Python.
5. Apply inheritance to model real-world hierarchies effectively.

11 Lab 13 & 14: Abstraction and Encapsulation

11.1 Objective

To understand and implement the object-oriented principles of **Abstraction** and **Encapsulation** in Python using private and protected variables, getter/setter methods, properties, and abstract base classes.

11.2 Theory

Encapsulation is one of the four fundamental principles of object-oriented programming. It refers to the bundling of data (attributes) and the methods that operate on that data within a single unit, or class, and restricting direct access to some of the object's components. This is done to protect the object's internal state from unintended interference and misuse.

Access Modifiers in Python: Unlike some other languages (Java, C++), Python does not enforce strict access control. Instead, it uses naming conventions:

- **Public** (`attribute`): Accessible from anywhere. No underscore prefix.
- **Protected** (`_attribute`): Intended for internal and subclass use only. Indicated by a single leading underscore.
- **Private** (`__attribute`): Accessible only within the class. Indicated by a double leading underscore. Python applies name mangling, transforming `__attr` into `_ClassName__attr`.

Getter and Setter Methods: Getter methods are used to retrieve the value of private/protected attributes. Setter methods are used to update those values, often with validation logic embedded. This pattern ensures data integrity.

Python Properties: The `@property` decorator provides a Pythonic way to implement getters. The `@attribute.setter` decorator provides a Pythonic way to implement setters. This approach allows attribute-style access while retaining control logic.

Abstraction refers to hiding complex implementation details and exposing only the essential features of an object. In Python, abstraction is achieved using the `abc` module (Abstract Base Classes). A class that inherits from `ABC` and contains at least one method decorated with `@abstractmethod` is an abstract class and **cannot be instantiated** directly.

Why Use Abstraction?

- It defines a contract: any subclass must implement all abstract methods.
- It reduces complexity by hiding irrelevant details from the user.
- It promotes a clean interface-driven design.

Difference Between Abstraction and Encapsulation:

- **Encapsulation** is about *how* data is stored and protected (data hiding mechanism).
- **Abstraction** is about *what* functionality is exposed (interface design).

Both principles work together to produce robust, maintainable, and secure Python programs.

11.3 Programs

11.3.1 Basic Programs

Program 1: Private Variables

Problem Statement: Write a Python program to demonstrate private variables using name mangling.

Algorithm:

Step 1: Define a class `BankAccount` with a private attribute `__balance`.
Step 2: Initialize `__balance` in the constructor.
Step 3: Define a method `show_balance()` to display the balance.
Step 4: Create an object and call the display method.
Step 5: Attempt to access the private variable directly.

```python
# Program 1: Private variables
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance  # private variable

    def show_balance(self):
        print(f"Owner: {self.owner}")
        print(f"Balance: Rs. {self.__balance:.2f}")

acc = BankAccount("Arjun", 25000)
acc.show_balance()

# Accessing via name mangling (not recommended)
print(f"\nAccess via mangling: Rs. {acc._BankAccount__balance:.2f
    }")
```

Sample Input: None (hardcoded)

Sample Output

```
Owner: Arjun
Balance: Rs. 25000.00

Access via mangling: Rs. 25000.00
```

Explanation: The `__balance` attribute is private. It cannot be accessed directly as `acc.__balance`, but Python allows access through the mangled name `acc._BankAccount__balance` for debugging purposes only.

Program 2: Getter and Setter

Problem Statement: Implement getter and setter methods to safely access and modify private attributes.

Algorithm:

Step 1: Define class `Student` with private attribute `__grade`.
Step 2: Implement `get_grade()` to return the grade.
Step 3: Implement `set_grade()` with validation (0–100).
Step 4: Create an object, set a grade, and display it.

```python
# Program 2: Getter and setter methods
class Student:
    def __init__(self, name):
        self.name = name
        self.__grade = 0  # private

    def get_grade(self):
        return self.__grade

    def set_grade(self, grade):
        if 0 <= grade <= 100:
            self.__grade = grade
            print(f"Grade set to {grade} for {self.name}.")
        else:
            print("Invalid grade. Must be between 0 and 100.")

s = Student("Kavya")
s.set_grade(88)
print(f"Grade: {s.get_grade()}")
s.set_grade(110)
s.set_grade(75)
print(f"Updated Grade: {s.get_grade()}")
```

Sample Input: None (hardcoded)

Sample Output

```
Grade set to 88 for Kavya.
Grade: 88
Invalid grade. Must be between 0 and 100.
Grade set to 75 for Kavya.
Updated Grade: 75
```

Explanation: The setter method includes a validation check. Invalid grades (above 100 or below 0) are rejected, protecting the object's state.

Program 3: Simple Encapsulation

Problem Statement: Demonstrate simple encapsulation by bundling data and related methods in one class.

Algorithm:

Step 1: Define class `Circle` with private attribute `__radius`.
Step 2: Provide a method to compute area and circumference.
Step 3: Protect radius from negative values using setter.
Step 4: Display the results.

```python
# Program 3: Simple encapsulation
class Circle:
    PI = 3.14159

    def __init__(self, radius):
        self.__radius = 0
        self.set_radius(radius)

    def set_radius(self, radius):
        if radius > 0:
            self.__radius = radius
        else:
            print("Radius must be positive.")

    def get_radius(self):
        return self.__radius

    def area(self):
        return Circle.PI * self.__radius ** 2

    def circumference(self):
        return 2 * Circle.PI * self.__radius

c = Circle(7)
print(f"Radius: {c.get_radius()}")
print(f"Area: {c.area():.2f}")
print(f"Circumference: {c.circumference():.2f}")

c.set_radius(-3)
print(f"Radius after invalid set: {c.get_radius()}")
```

Sample Input: None (hardcoded)

Sample Output

```
Radius: 7
Area: 153.94
Circumference: 43.98
Radius must be positive.
Radius after invalid set: 7
```

Explanation: The radius is protected from negative values. The object maintains a valid internal state at all times.

Program 4: Protected Variables

Problem Statement: Demonstrate the use of protected variables (single underscore) and their inheritance behavior.

Algorithm:

Step 1: Define class `Employee` with a protected attribute `_salary`.
Step 2: Define a subclass `Manager` that accesses and modifies `_salary`.
Step 3: Display salary from both parent and child context.

```python
# Program 4: Protected variables
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self._salary = salary  # protected

    def display(self):
        print(f"Employee: {self.name}")
        print(f"Salary: Rs. {self._salary:.2f}")

class Manager(Employee):
    def __init__(self, name, salary, bonus):
        super().__init__(name, salary)
        self._bonus = bonus  # protected

    def display(self):
        super().display()
        print(f"Bonus: Rs. {self._bonus:.2f}")
        print(f"Total CTC: Rs. {self._salary + self._bonus:.2f}")

e = Employee("Ravi", 40000)
m = Manager("Priya", 80000, 20000)

print("--- Employee ---")
e.display()
print("--- Manager ---")
m.display()
```

Sample Input: None (hardcoded)

Sample Output

```
--- Employee ---
Employee: Ravi
Salary: Rs. 40000.00
--- Manager ---
Employee: Priya
Salary: Rs. 80000.00
Bonus: Rs. 20000.00
Total CTC: Rs. 100000.00
```

Explanation: Protected attributes are accessible within the class and by subclasses. The Manager subclass directly uses the inherited `_salary` attribute.

Program 5: Basic Abstraction

Problem Statement: Demonstrate basic abstraction using the `abc` module to define an abstract class and implement it in a concrete subclass.

Algorithm:

Step 1: Import `ABC` and `abstractmethod` from `abc`.
Step 2: Define abstract class `Shape` with abstract method `area()`.
Step 3: Create concrete subclass `Rectangle` implementing `area()`.
Step 4: Create object of `Rectangle` and invoke `area()`.

```python
# Program 5: Basic abstraction using ABC
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

    def describe(self):
        print(f"Shape: {self.__class__.__name__}")

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)

r = Rectangle(8, 5)
r.describe()
print(f"Area: {r.area()}")
print(f"Perimeter: {r.perimeter()}")
```

Sample Input: None (hardcoded)

Sample Output

```
Shape: Rectangle
```

```
Area: 40
Perimeter: 26
```

Explanation: `Shape` is abstract and cannot be instantiated. `Rectangle` provides concrete implementations of all abstract methods.

11.3.2 Medium Programs

Program 6: Bank System with Encapsulation

```python
# Program 6: Bank system with encapsulation
class BankAccount:
    def __init__(self, account_no, owner, balance=0):
        self.__account_no = account_no
        self.__owner = owner
        self.__balance = balance
        self.__transactions = []

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            self.__transactions.append(f"Deposited: Rs. {amount
                :.2f}")
            print(f"Deposited Rs. {amount:.2f}. New Balance: Rs.
                {self.__balance:.2f}")
        else:
            print("Invalid deposit amount.")

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            self.__transactions.append(f"Withdrawn: Rs. {amount
                :.2f}")
            print(f"Withdrawn Rs. {amount:.2f}. Remaining: Rs. {
                self.__balance:.2f}")
        elif amount > self.__balance:
            print("Insufficient funds.")
        else:
            print("Invalid withdrawal amount.")

    def get_balance(self):
        return self.__balance

    def print_statement(self):
        print(f"\n=== Account Statement ===")
        print(f"Account: {self.__account_no} | Owner: {self.
            __owner}")
        for t in self.__transactions:
            print(f"  {t}")
        print(f"Current Balance: Rs. {self.__balance:.2f}")

acc = BankAccount("SB1001", "Dr. Babu", 10000)
```

```
acc.deposit(5000)
acc.withdraw(3000)
acc.withdraw(20000)
acc.deposit(-500)
acc.print_statement()
```

Sample Output

```
Deposited Rs. 5000.00. New Balance: Rs. 15000.00
Withdrawn Rs. 3000.00. Remaining: Rs. 12000.00
Insufficient funds.
Invalid deposit amount.

=== Account Statement ===
Account: SB1001 | Owner: Dr. Babu
  Deposited: Rs. 5000.00
  Withdrawn: Rs. 3000.00
Current Balance: Rs. 12000.00
```

Program 7: Student Grade Validation

```python
# Program 7: Student grade validation using properties
class Student:
    def __init__(self, name, roll_no):
        self.name = name
        self.roll_no = roll_no
        self.__marks = {}

    def add_subject(self, subject, marks):
        if 0 <= marks <= 100:
            self.__marks[subject] = marks
            print(f"Added {subject}: {marks}/100")
        else:
            print(f"Invalid marks for {subject}. Must be 0-100.")

    @property
    def total(self):
        return sum(self.__marks.values())

    @property
    def average(self):
        if self.__marks:
            return self.total / len(self.__marks)
        return 0

    @property
    def grade(self):
        avg = self.average
        if avg >= 90: return "O (Outstanding)"
        elif avg >= 80: return "A+"
        elif avg >= 70: return "A"
```

```
        elif avg >= 60: return "B+"
        elif avg >= 50: return "B"
        else: return "F (Fail)"

    def display(self):
        print(f"\nStudent: {self.name} | Roll: {self.roll_no}")
        for sub, m in self.__marks.items():
            print(f"  {sub}: {m}/100")
        print(f"Total: {self.total} | Average: {self.average:.2f}
            | Grade: {self.grade}")

s = Student("Sneha", "1CR23CS010")
s.add_subject("Python", 91)
s.add_subject("DBMS", 85)
s.add_subject("OS", 78)
s.add_subject("CN", 110)
s.display()
```

Sample Output

```
Added Python: 91/100
Added DBMS: 85/100
Added OS: 78/100
Invalid marks for CN. Must be 0-100.
Student: Sneha | Roll: 1CR23CS010
  Python: 91/100
  DBMS: 85/100
  OS: 78/100
Total: 254 | Average: 84.67 | Grade: A+
```

Program 8: Car Speed Control

```
# Program 8: Car speed control using encapsulation
class Car:
    MAX_SPEED = 200  # class constant

    def __init__(self, brand, model):
        self.brand = brand
        self.model = model
        self.__speed = 0
        self.__fuel = 100.0

    @property
    def speed(self):
        return self.__speed

    def accelerate(self, increment):
        new_speed = self.__speed + increment
        if new_speed <= Car.MAX_SPEED and self.__fuel > 0:
            self.__speed = new_speed
            self.__fuel -= increment * 0.05
```

```
            print(f"Speed: {self.__speed} km/h | Fuel: {self.
                __fuel:.1f}L")
        elif self.__fuel <= 0:
            print("Out of fuel!")
        else:
            print(f"Speed limit reached! Max: {Car.MAX_SPEED} km/
                h")

    def brake(self, decrement):
        self.__speed = max(0, self.__speed - decrement)
        print(f"Braking... Speed: {self.__speed} km/h")

    def status(self):
        print(f"\n{self.brand} {self.model}: Speed={self.__speed}
            km/h | Fuel={self.__fuel:.1f}L")

car = Car("Honda", "City")
car.accelerate(60)
car.accelerate(80)
car.accelerate(90)
car.brake(40)
car.status()
```

Sample Output

```
Speed: 60 km/h | Fuel: 97.0L
Speed: 140 km/h | Fuel: 93.0L
Speed limit reached! Max: 200 km/h
Braking... Speed: 100 km/h

Honda City: Speed=100 km/h | Fuel=93.0L
```

Program 9: Data Validation

```
# Program 9: Data validation using properties
class Employee:
    def __init__(self, name, age, salary):
        self.name = name
        self.age = age          # uses property setter
        self.salary = salary    # uses property setter

    @property
    def age(self):
        return self.__age

    @age.setter
    def age(self, value):
        if 18 <= value <= 65:
            self.__age = value
        else:
            raise ValueError(f"Age {value} is invalid. Must be
```

```
                between 18 and 65.")

    @property
    def salary(self):
        return self.__salary

    @salary.setter
    def salary(self, value):
        if value >= 15000:
            self.__salary = value
        else:
            raise ValueError("Salary must be at least Rs. 15,000.
               ")

    def display(self):
        print(f"Name: {self.name} | Age: {self.__age} | Salary:
           Rs. {self.__salary:.2f}")

try:
    e1 = Employee("Arjun", 28, 45000)
    e1.display()
    e2 = Employee("Meena", 17, 30000)
except ValueError as e:
    print(f"Error: {e}")

try:
    e3 = Employee("Suresh", 35, 5000)
except ValueError as e:
    print(f"Error: {e}")
```

Sample Output

```
Name: Arjun | Age: 28 | Salary: Rs. 45000.00
Error: Age 17 is invalid. Must be between 18 and 65.
Error: Salary must be at least Rs. 15,000.
```

Program 10: Abstract Class Example

```
# Program 10: Abstract class with multiple concrete
   implementations
from abc import ABC, abstractmethod

class Animal(ABC):
    def __init__(self, name, sound):
        self.name = name
        self.sound = sound

    @abstractmethod
    def speak(self):
        pass
```

```
    @abstractmethod
    def move(self):
        pass

    def describe(self):
        print(f"Animal: {self.name}")

class Dog(Animal):
    def speak(self):
        print(f"{self.name} says: {self.sound}!")

    def move(self):
        print(f"{self.name} runs on four legs.")

class Bird(Animal):
    def speak(self):
        print(f"{self.name} sings: {self.sound}!")

    def move(self):
        print(f"{self.name} flies with wings.")

class Fish(Animal):
    def speak(self):
        print(f"{self.name} makes bubbles: {self.sound}")

    def move(self):
        print(f"{self.name} swims with fins.")

animals = [Dog("Rex", "Woof"), Bird("Sparrow", "Tweet"), Fish("
   Nemo", "Blub")]
for animal in animals:
    animal.describe()
    animal.speak()
    animal.move()
    print()
```

Sample Output

```
Animal: Rex
Rex says: Woof!
Rex runs on four legs.

Animal: Sparrow
Sparrow sings: Tweet!
Sparrow flies with wings.

Animal: Nemo
Nemo makes bubbles: Blub
Nemo swims with fins.
```

11.3.3 Advanced Programs

Program 11: ATM System Using Encapsulation

```
# Program 11: ATM system using encapsulation
class ATM:
    __DAILY_LIMIT = 50000

    def __init__(self, card_number, pin, balance):
        self.__card_number = card_number
        self.__pin = pin
        self.__balance = balance
        self.__authenticated = False
        self.__withdrawn_today = 0
        self.__log = []

    def __authenticate(self, pin):
        if pin == self.__pin:
            self.__authenticated = True
            print("Authentication successful.")
            return True
        else:
            print("Incorrect PIN. Access denied.")
            return False

    def __mask_card(self):
        num = str(self.__card_number)
        return "*" * (len(num) - 4) + num[-4:]

    def login(self, pin):
        return self.__authenticate(pin)

    def check_balance(self):
        if not self.__authenticated:
            print("Please authenticate first.")
            return
        print(f"Card: {self.__mask_card()} | Balance: Rs. {self.
           __balance:.2f}")

    def withdraw(self, amount):
        if not self.__authenticated:
            print("Please authenticate first.")
            return
        if amount <= 0:
            print("Invalid amount.")
        elif amount > self.__balance:
            print("Insufficient balance.")
        elif self.__withdrawn_today + amount > ATM.__DAILY_LIMIT:
            remaining = ATM.__DAILY_LIMIT - self.
               __withdrawn_today
            print(f"Daily limit exceeded. You can withdraw up to
               Rs. {remaining:.2f} today.")
```

```
        else:
            self.__balance -= amount
            self.__withdrawn_today += amount
            self.__log.append(f"Withdrawn Rs. {amount:.2f}")
            print(f"Please collect Rs. {amount:.2f}. Balance: Rs.
                {self.__balance:.2f}")

    def deposit(self, amount):
        if not self.__authenticated:
            print("Please authenticate first.")
            return
        if amount > 0:
            self.__balance += amount
            self.__log.append(f"Deposited Rs. {amount:.2f}")
            print(f"Deposited Rs. {amount:.2f}. Balance: Rs. {
               self.__balance:.2f}")
        else:
            print("Invalid deposit amount.")

    def print_log(self):
        if not self.__authenticated:
            print("Please authenticate first.")
            return
        print("\n=== Transaction Log ===")
        if not self.__log:
            print("No transactions yet.")
        for entry in self.__log:
            print(f"  - {entry}")

    def logout(self):
        self.__authenticated = False
        print("Logged out successfully.")

atm = ATM("1234567890123456", 1234, 75000)
atm.check_balance()
atm.login(9999)
atm.login(1234)
atm.check_balance()
atm.withdraw(10000)
atm.withdraw(40000)
atm.deposit(5000)
atm.print_log()
atm.logout()
atm.withdraw(1000)
```

Sample Output

```
Please authenticate first.
Incorrect PIN. Access denied.
Authentication successful.
Card: ************3456 | Balance: Rs. 75000.00
Please collect Rs. 10000.00. Balance: Rs. 65000.00
```

```
Daily limit exceeded. You can withdraw up to Rs. 40000.00 today.
Deposited Rs. 5000.00. Balance: Rs. 70000.00

=== Transaction Log ===
  - Withdrawn Rs. 10000.00
  - Deposited Rs. 5000.00
Logged out successfully.
Please authenticate first.
```

Program 12: Payment System Using Abstraction

```python
# Program 12: Payment system using abstraction
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    def __init__(self, gateway_name):
        self.__gateway_name = gateway_name
        self._transaction_log = []

    @property
    def gateway_name(self):
        return self.__gateway_name

    @abstractmethod
    def authenticate(self, credentials):
        pass

    @abstractmethod
    def process_payment(self, amount, recipient):
        pass

    @abstractmethod
    def get_fee(self, amount):
        pass

    def complete_transaction(self, amount, recipient, credentials
       ):
        print(f"\n[{self.gateway_name}] Initiating payment...")
        if not self.authenticate(credentials):
            print(f"[{self.gateway_name}] Authentication failed.
               Aborting.")
            return
        fee = self.get_fee(amount)
        total = amount + fee
        print(f"  Amount: Rs. {amount:.2f} | Fee: Rs. {fee:.2f} |
            Total: Rs. {total:.2f}")
        result = self.process_payment(amount, recipient)
        self._transaction_log.append({
            "gateway": self.gateway_name,
            "to": recipient,
            "amount": amount,
```

```
            "fee": fee,
            "status": "Success" if result else "Failed"
        })
        if result:
            print(f"[{self.gateway_name}] Payment of Rs. {amount
                :.2f} to {recipient} successful.")

    def print_log(self):
        print(f"\n=== {self.gateway_name} Transactions ===")
        for t in self._transaction_log:
            print(f"  To: {t['to']} | Amount: Rs. {t['amount']:.2
                f} | Status: {t['status']}")

class UPIPayment(PaymentGateway):
    def __init__(self, upi_id, pin):
        super().__init__("UPI")
        self.__upi_id = upi_id
        self.__pin = pin

    def authenticate(self, credentials):
        if credentials.get("pin") == self.__pin:
            print(f"  UPI ID: {self.__upi_id} authenticated.")
            return True
        return False

    def get_fee(self, amount):
        return 0.0  # UPI is free

    def process_payment(self, amount, recipient):
        print(f"  Sending Rs. {amount:.2f} via UPI to {recipient}
            @upi...")
        return True

class NetBanking(PaymentGateway):
    def __init__(self, user_id, password):
        super().__init__("Net Banking")
        self.__user_id = user_id
        self.__password = password

    def authenticate(self, credentials):
        if (credentials.get("user_id") == self.__user_id and
                credentials.get("password") == self.__password):
            print(f"  Net Banking user {self.__user_id} verified.
                ")
            return True
        return False

    def get_fee(self, amount):
        return 25.0 if amount < 10000 else 50.0

    def process_payment(self, amount, recipient):
```

```
        print(f"  Transferring Rs. {amount:.2f} via NEFT to {
           recipient}...")
        return True

upi = UPIPayment("student@okaxis", 4321)
upi.complete_transaction(1500, "Amazon", {"pin": 4321})
upi.complete_transaction(500, "Zomato", {"pin": 0000})

nb = NetBanking("user001", "secure@123")
nb.complete_transaction(8000, "LIC Premium", {"user_id": "user001
   ", "password": "secure@123"})
nb.complete_transaction(25000, "EMI Payment", {"user_id": "
   user001", "password": "secure@123"})

upi.print_log()
nb.print_log()
```

Sample Output

```
[UPI] Initiating payment...
  UPI ID: student@okaxis authenticated.
  Amount: Rs. 1500.00 | Fee: Rs. 0.00 | Total: Rs. 1500.00
  Sending Rs. 1500.00 via UPI to Amazon@upi...
[UPI] Payment of Rs. 1500.00 to Amazon successful.

[UPI] Initiating payment...
[UPI] Authentication failed. Aborting.

[Net Banking] Initiating payment...
  Net Banking user user001 verified.
  Amount: Rs. 8000.00 | Fee: Rs. 25.00 | Total: Rs. 8025.00
  Transferring Rs. 8000.00 via NEFT to LIC Premium...
[Net Banking] Payment of Rs. 8000.00 to LIC Premium successful.

[Net Banking] Initiating payment...
  Net Banking user user001 verified.
  Amount: Rs. 25000.00 | Fee: Rs. 50.00 | Total: Rs. 25050.00
  Transferring Rs. 25000.00 via NEFT to EMI Payment...
[Net Banking] Payment of Rs. 25000.00 to EMI Payment successful.

=== UPI Transactions ===
  To: Amazon | Amount: Rs. 1500.00 | Status: Success

=== Net Banking Transactions ===
  To: LIC Premium | Amount: Rs. 8000.00 | Status: Success
  To: EMI Payment | Amount: Rs. 25000.00 | Status: Success
```

11.4 Learning Outcomes

Upon completing this lab, students will be able to:

1. Implement encapsulation using private, protected, and public access modifiers.

2. Design getter and setter methods to safely access private attributes.
3. Use Python's `@property` and `@setter` decorators for Pythonic data access.
4. Create abstract base classes using the `abc` module.
5. Apply abstraction to define interfaces that enforce method contracts in subclasses.
6. Build real-world applications such as ATM and payment systems using OOP principles.

— End of Laboratory Manual —

CMR University – School of Engineering and Technology

Department of Computer Science and Engineering

Academic Year 2025–2026

Prepared by Dr. Pandava Sudharshan Babu and Python Faculty Team

CMR University, Bengaluru – 562149